Bayes' rule: update what you believe, using data

$$p(A \mid B) = \frac{p(B \mid A)\, p(A)}{p(B)}$$

> Bayes' rule: what to believe after seeing B, from what we believed before and how well each belief explains B.

Bayes' rule: update what you believe, using data

$$p(A \mid B) = \frac{p(B \mid A)\, p(A)}{p(B)}$$

$$\underbrace{p(A \mid B)}_{\text{posterior}} = \frac{\overbrace{p(B \mid A)}^{\text{likelihood}}\ \overbrace{p(A)}^{\text{prior}}}{\underbrace{p(B)}_{\text{evidence}}}$$

> Four names for the whole lecture: posterior, likelihood, prior and evidence.

In machine learning, the unknown is the parameters

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)\, p(\theta)}{p(\mathcal{D})}$$

A becomes θ, every weight and bias of the model; B becomes $\mathcal{D}$, the training data.

In machine learning, the unknown is the parameters

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)\, p(\theta)}{p(\mathcal{D})}$$

Usual training

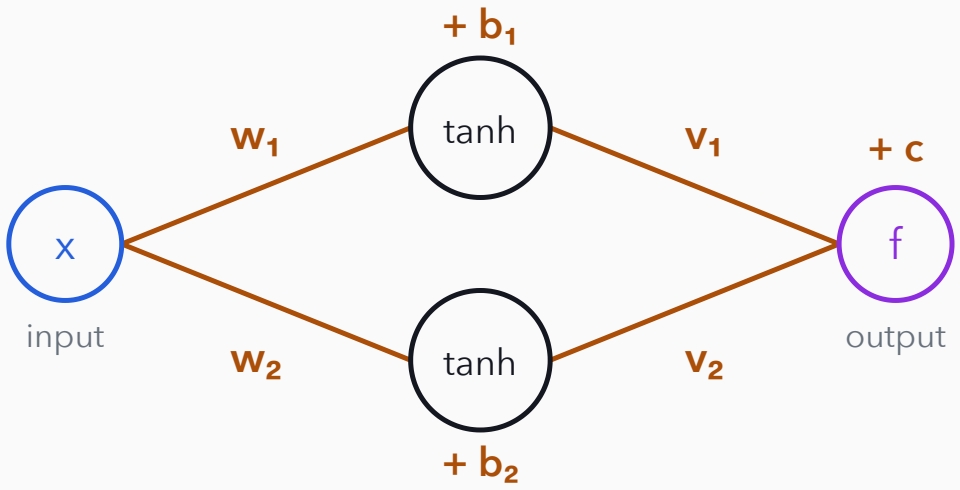
keeps one θ: the best fit to the data $\mathcal{D}$.

Bayesian learning

keeps every θ, weighted by $p(\theta \mid \mathcal{D})$.

> Usual training keeps one θ. The Bayesian answer keeps a whole distribution over θ.

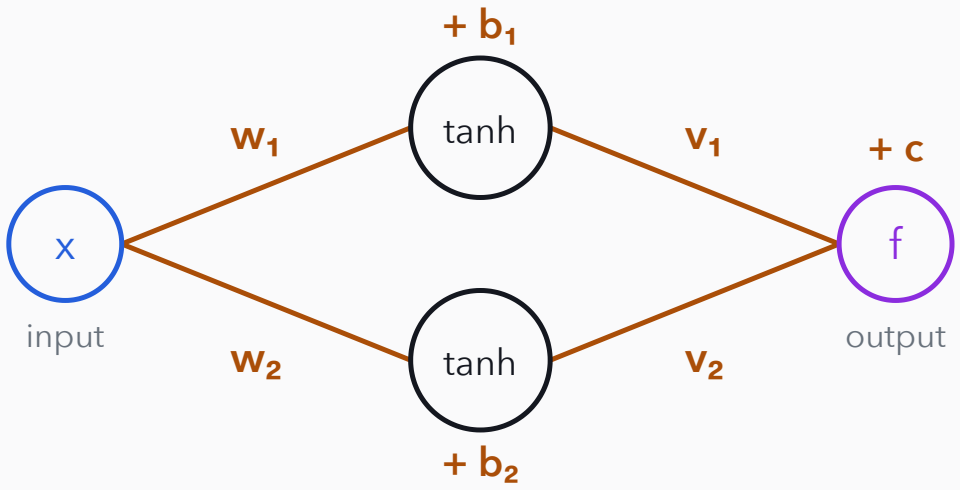
A tiny network: one input, two hidden units, one output



$$f_\theta(x) = v_1 \tanh(w_1 x + b_1) + v_2 \tanh(w_2 x + b_2) + c$$

Our running example: one input x, two tanh hidden units and one output f.

A tiny network: one input, two hidden units, one output



$$f_\theta(x) = v_1 \tanh(w_1 x + b_1) + v_2 \tanh(w_2 x + b_2) + c$$

$$\theta = (w_1, w_2, b_1, b_2, v_1, v_2, c) \in \mathbb{R}^7$$

Seven unknown numbers: four weights and three biases.

> Every weight and bias is unknown, so θ is seven numbers.

A prior on every parameter

$$\theta_j \sim \mathcal{N}(0, 1), \quad j = 1, \ldots, 7$$

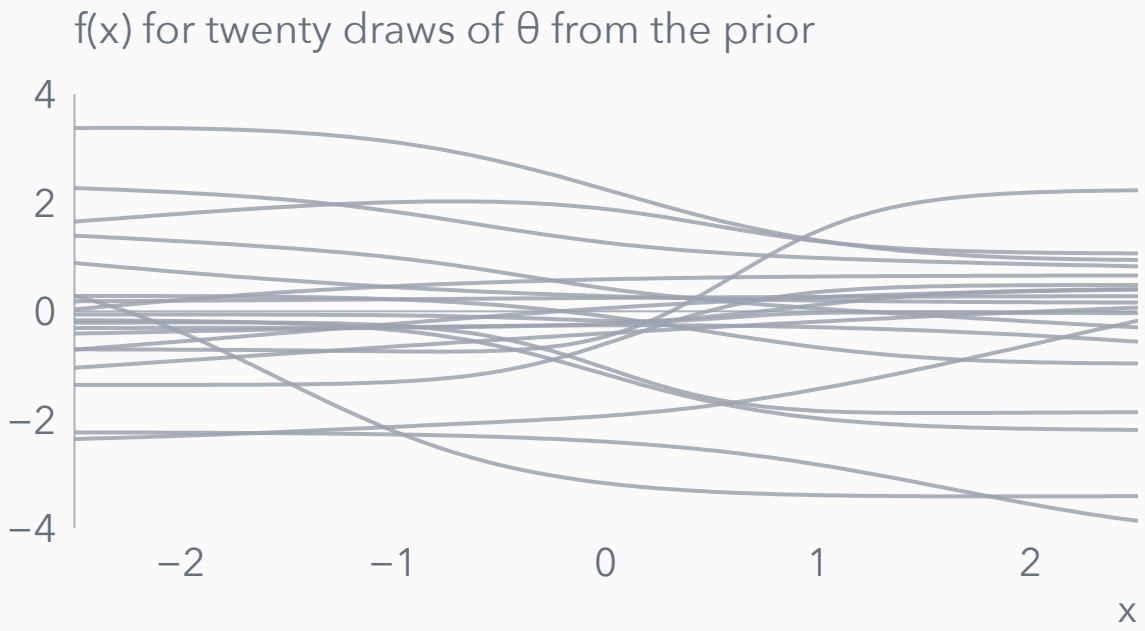
$$p(\theta) = \prod_{j=1}^{7} \mathcal{N}(\theta_j;\, 0,\, 1)$$

Before seeing data, every parameter gets its own prior: $\theta_j \sim N(0, 1)$, independently.

A prior on every parameter

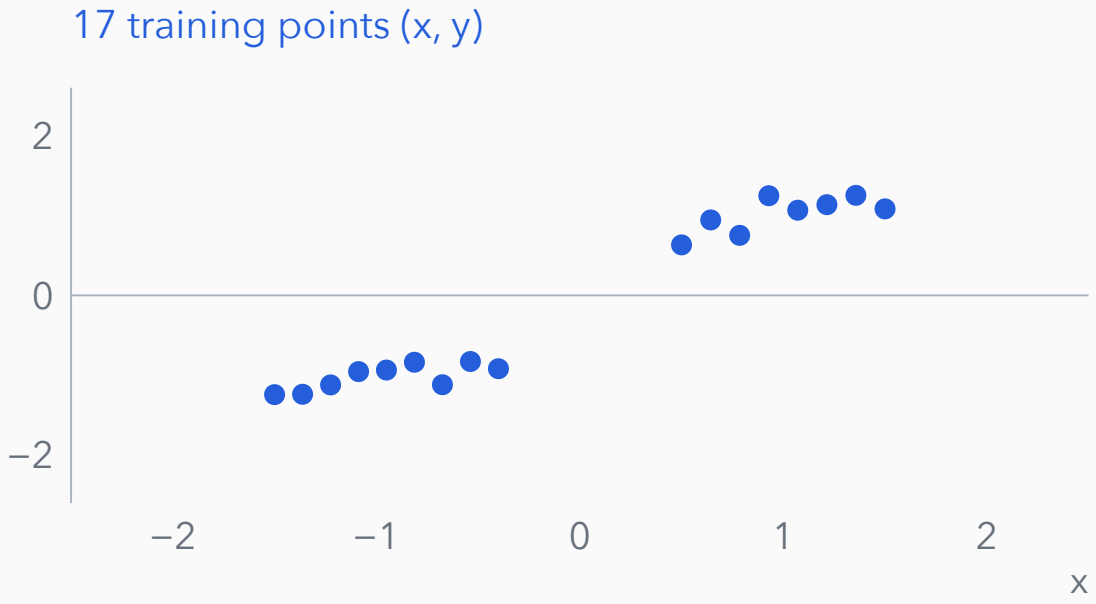
$$\theta_j \sim \mathcal{N}(0, 1), \quad j = 1, \ldots, 7$$

$$p(\theta) = \prod_{j=1}^{7} \mathcal{N}(\theta_j;\, 0,\, 1)$$



Each draw of θ is a whole function, so the prior is a distribution over functions. Here are twenty.

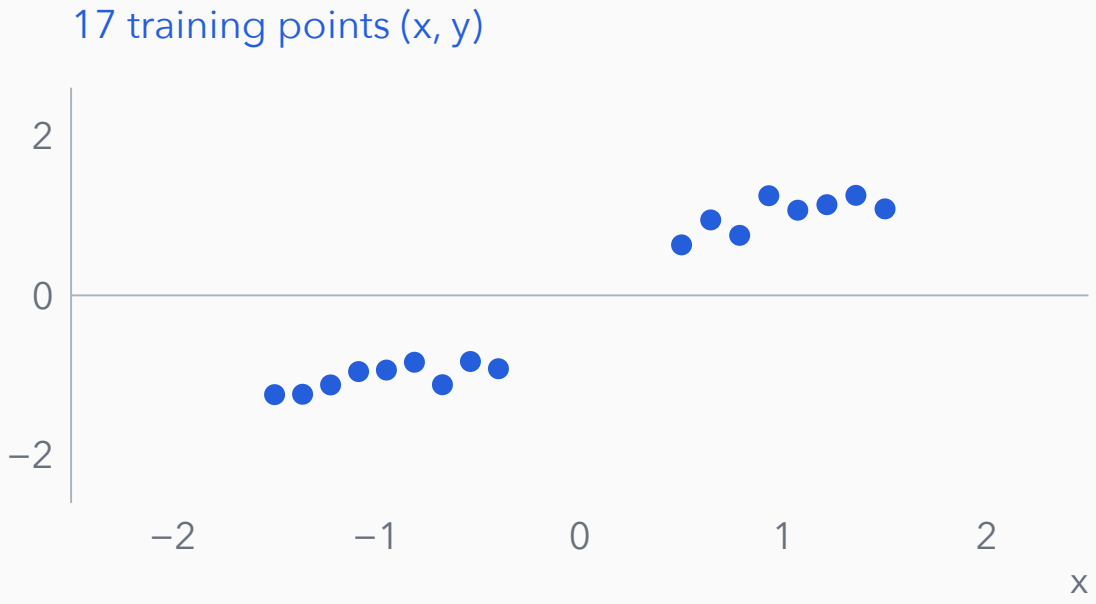
The data, and how well one θ explains them



$$y_i \mid x_i, \theta \sim \mathcal{N}\big(f_\theta(x_i),\, 0.1^2\big)$$

> Seventeen training points with a gap in the middle. Each y is the network's output plus Gaussian noise of size 0.1.

The data, and how well one θ explains them



$$y_i \mid x_i, \theta \sim \mathcal{N}\big(f_\theta(x_i),\, 0.1^2\big)$$

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{17} \mathcal{N}\big(y_i;\, f_\theta(x_i),\, 0.1^2\big)$$

> The likelihood of θ multiplies all seventeen densities: how well this one θ explains the whole dataset.

The right-hand side, term by term, for one θ

$$\log p(\mathcal{D} \mid \theta_A) = \sum_{i=1}^{17} \log \mathcal{N}\left(y_i;\ f_{\theta_A}(x_i),\ 0.1^2\right)$$

$$= \underbrace{1.16}_{i=1} + \underbrace{0.95}_{i=2} + \cdots + \underbrace{0.83}_{i=17} = 12.17$$

The likelihood: one log density per data point, seventeen terms, added up.

The right-hand side, term by term, for one θ

$$\log p(\mathcal{D} \mid \theta_A) = \sum_{i=1}^{17} \log \mathcal{N}\big(y_i;\ f_{\theta_A}(x_i),\ 0.1^2\big)$$

$$= \underbrace{1.16}_{i=1} + \underbrace{0.95}_{i=2} + \cdots + \underbrace{0.83}_{i=17} = 12.17$$

$$\log p(\theta_A) = \sum_{j=1}^{7} \log \mathcal{N}(\theta_j; 0, 1) = \underbrace{-1.04}_{w_1} + \underbrace{-2.93}_{w_2} + \cdots + \underbrace{-0.93}_{c} = -9.41$$

The prior: one log density per parameter, seven terms, added up.

The right-hand side, term by term, for one θ

$$\log p(\mathcal{D} \mid \theta_A) = \sum_{i=1}^{17} \log \mathcal{N}\big(y_i;\ f_{\theta_A}(x_i),\ 0.1^2\big)$$

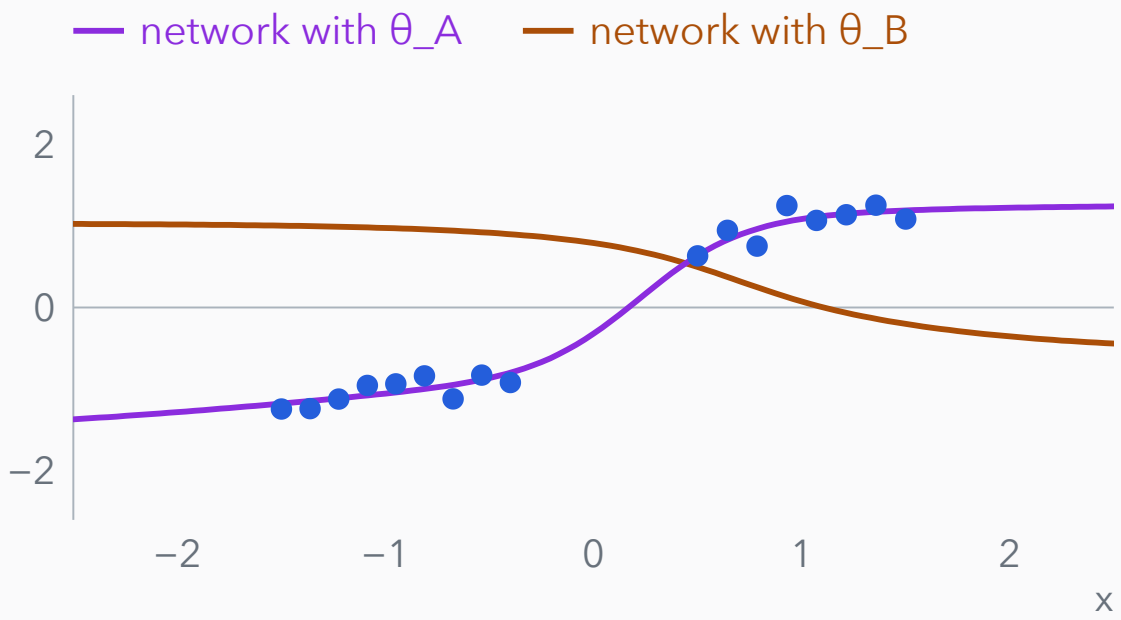
$$= \underbrace{1.16}_{i=1} + \underbrace{0.95}_{i=2} + \cdots + \underbrace{0.83}_{i=17} = 12.17$$

$$\log p(\theta_A) = \sum_{j=1}^{7} \log \mathcal{N}(\theta_j; 0, 1) = \underbrace{-1.04}_{w_1} + \underbrace{-2.93}_{w_2} + \cdots + \underbrace{-0.93}_{c} = -9.41$$

$$\log p(\mathcal{D}) = \log \int_{\mathbb{R}^7} p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta = \ \textbf{?}$$

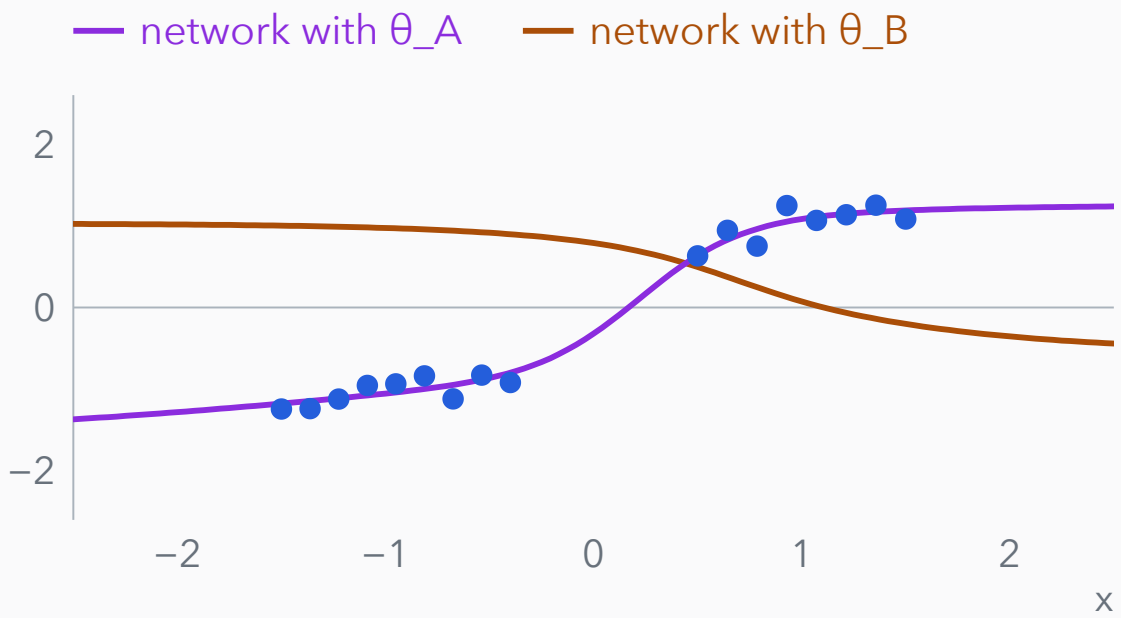
> The evidence needs every θ at once: an integral over all of $\mathbb{R}^7$.

Two settings of θ, side by side



θ_A follows the data. θ_B, one random draw from the prior, does not.

Two settings of θ, side by side



	θ_A	θ_B, a prior draw
$\log p(\mathcal{D} \mid \theta)$	12.2	−2,157.5
$\log p(\theta)$	−9.4	−9.9
sum of the two	2.8	−2,167.4

> Their priors are similar; their likelihoods differ by over 2,000 nats. A ratio of two posteriors does not even need p($\mathcal{D}$).

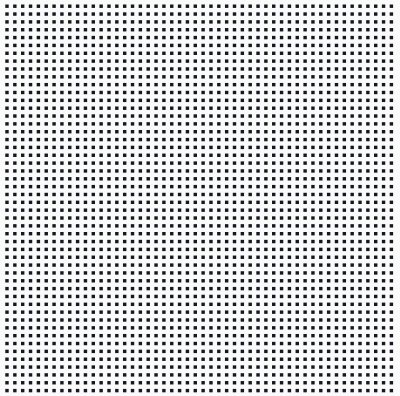
The evidence needs every θ, and has no formula

$$p(\mathcal{D}) = \int_{\mathbb{R}^7} p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta$$

1 unknown

100 points

2 unknowns



100 × 100 = 10,000 points

> θ enters through tanh, so this integral has no formula. A grid needs about 100 points for each parameter.

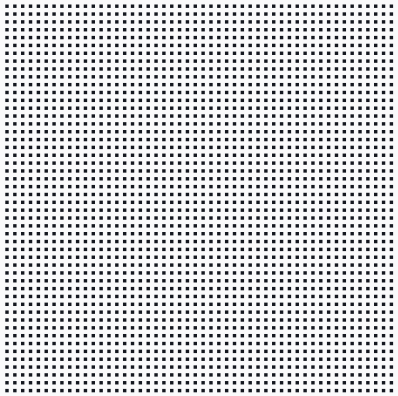
The evidence needs every θ, and has no formula

$$p(\mathcal{D}) = \int_{\mathbb{R}^7} p(\mathcal{D} \mid \theta)\, p(\theta)\, d\theta$$

1 unknown

100 points

2 unknowns

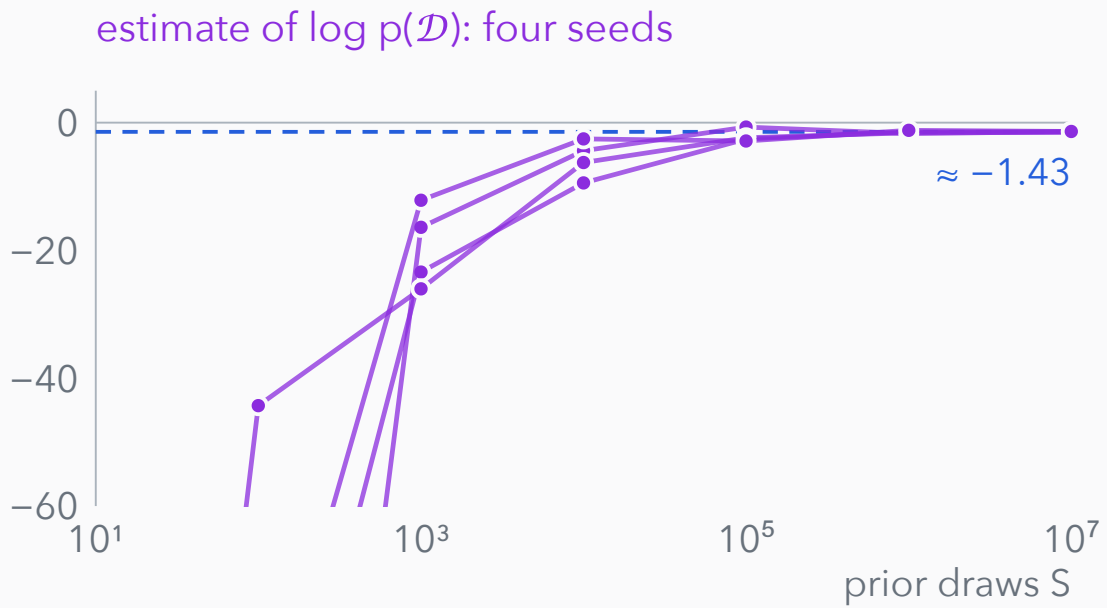


100 × 100 = 10,000 points

parameters	grid points
1	100
2	10,000
7, our network	10^{14}
10^6, a small real network	$10^{2,000,000}$

> Our seven parameters already need 10^{14} grid points; a real network needs 10^2,000,000.

Sampling from the prior: a few lucky draws carry everything

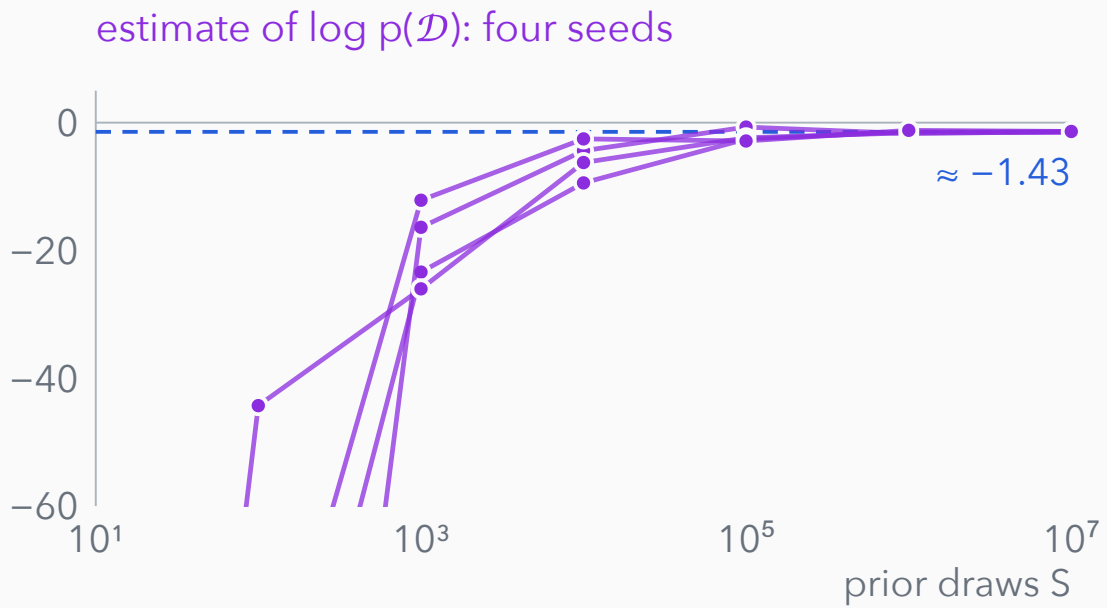


$$p(\mathcal{D}) \approx \frac{1}{S}\sum_{s=1}^{S} p(\mathcal{D} \mid \theta_s)$$

$$\theta_s \sim p(\theta)$$

Average the likelihood over draws from the prior: four seeds, up to ten million draws each.

Sampling from the prior: a few lucky draws carry everything



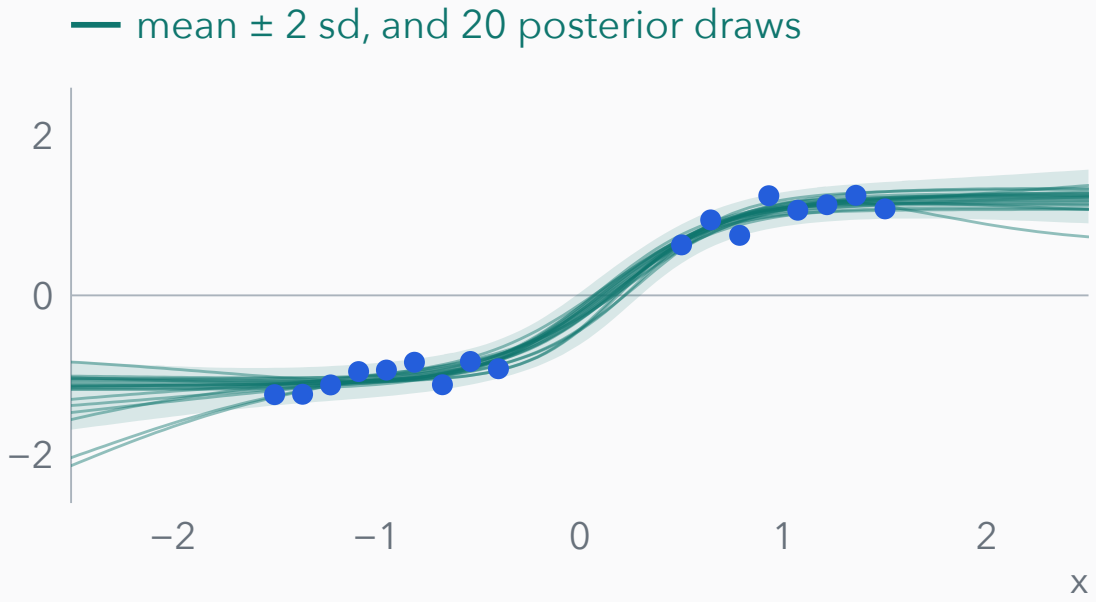
$$p(\mathcal{D}) \approx \frac{1}{S}\sum_{s=1}^{S} p(\mathcal{D} \mid \theta_s)$$

$$\theta_s \sim p(\theta)$$

Ten draws: off by about 1,000 nats. Ten million: about 103 draws carry the estimate, −1.43.

> Almost every prior draw explains the data terribly. Seven parameters just about work; more parameters or more data make it hopeless.

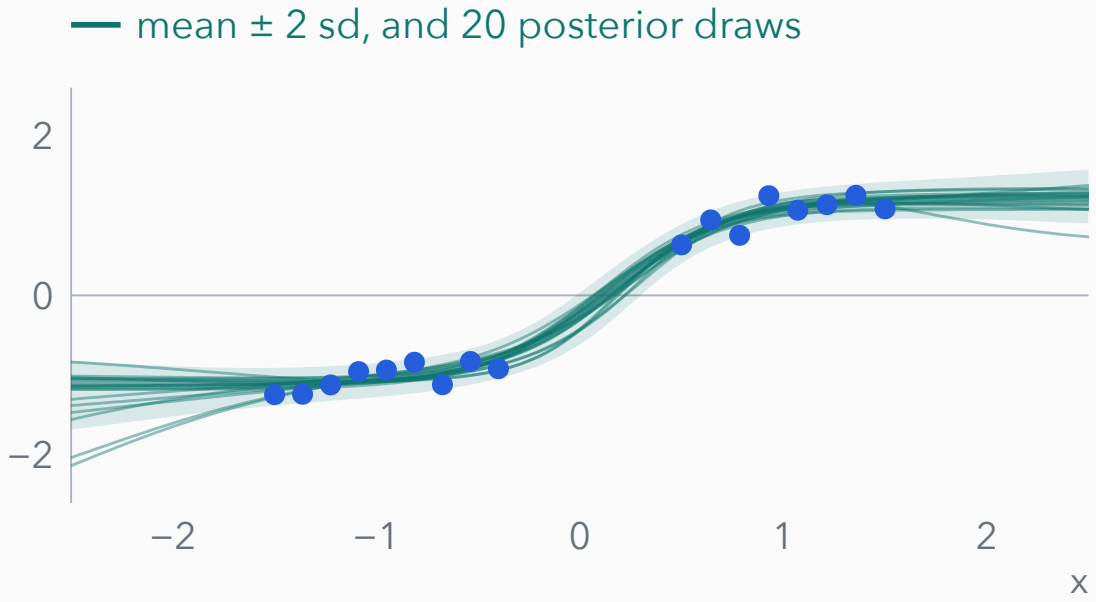
What we want in the end: predictions with error bars



$$p(y^* \mid x^*, \mathcal{D}) = \int p(y^* \mid x^*, \theta) \times p(\theta \mid \mathcal{D})\, d\theta$$

> The posterior is a means to an end. The posterior predictive averages every network's prediction at a new x*.

What we want in the end: predictions with error bars



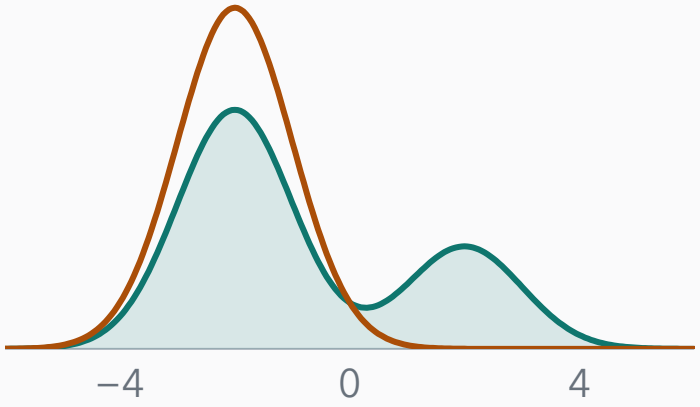
$$p(y^* \mid x^*, \mathcal{D}) = \int p(y^* \mid x^*, \theta) \times p(\theta \mid \mathcal{D})\, d\theta$$

Average every network's prediction, weighted by the posterior. The band widens where there are no data.

> Here it comes from a long MCMC run, used only as a reference. The band widens beyond the data and in the gap.

Three ways to approximate a posterior

Laplace

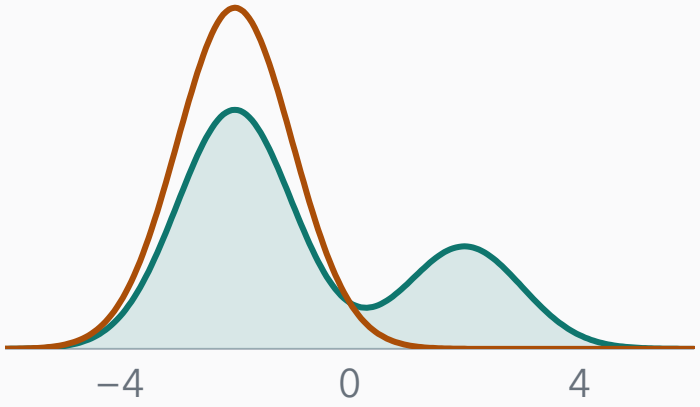


A Gaussian at the highest peak: $N(-2.0, 1.0^2)$. It never sees the second bump.

> Laplace: fit a Gaussian to the curvature at the highest peak.

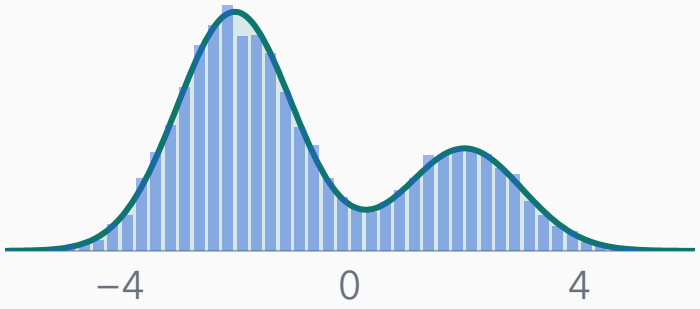
Three ways to approximate a posterior

Laplace



A Gaussian at the highest peak: N(−2.0, 1.0²). It never sees the second bump.

MCMC

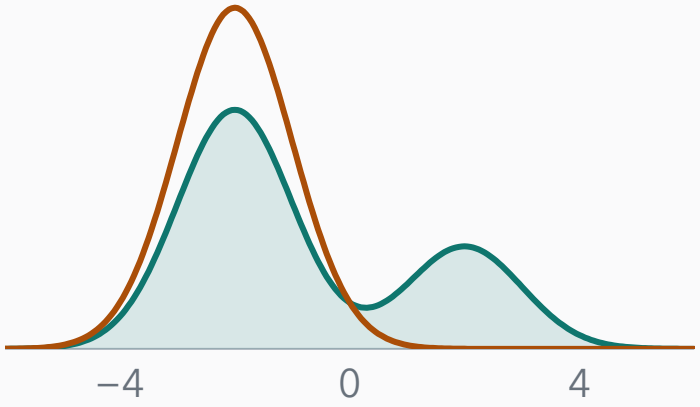


20,000 random-walk steps. Their mean, −0.80, matches the true −0.80.

> MCMC: a random walk whose visits follow the posterior. Right in the long run, but slow.

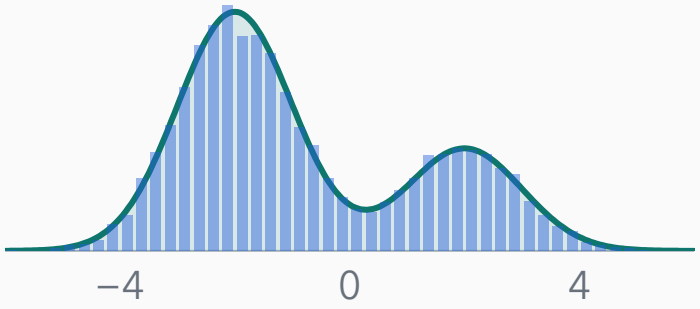
Three ways to approximate a posterior

Laplace



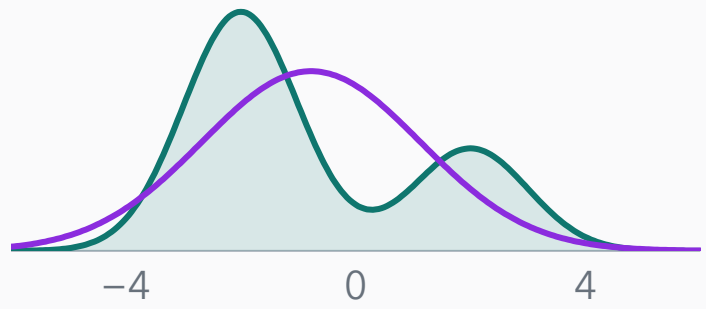
A Gaussian at the highest peak: N(−2.0, 1.0²). It never sees the second bump.

MCMC



20,000 random-walk steps. Their mean, −0.80, matches the true −0.80.

Variational inference



The closest Gaussian, found by optimisation: N(−0.78, 1.90²).

> Variational inference: choose the closest member of a simple family, by optimisation. That is today.

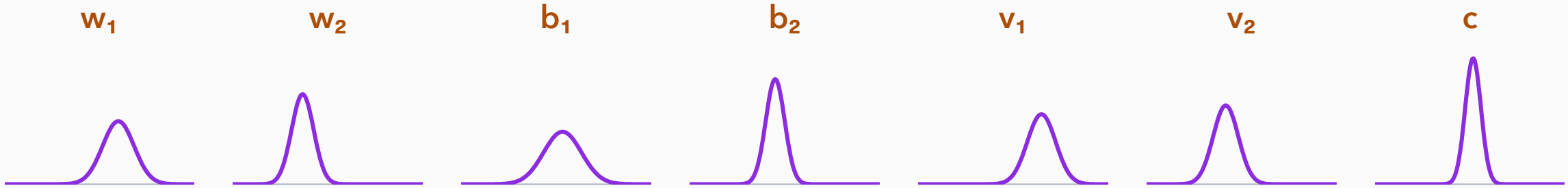
Variational inference: choose q, make it close to the posterior

$$q_\psi(\theta) = \prod_{j=1}^{7} \mathcal{N}(\theta_j;\, \mu_j,\, \sigma_j^2), \qquad \psi = (\mu_1, \ldots, \mu_7,\, \sigma_1, \ldots, \sigma_7)$$

> Pick a simple family: one Gaussian per parameter, each with its own mean and width. ψ is fourteen numbers.

Variational inference: choose q, make it close to the posterior

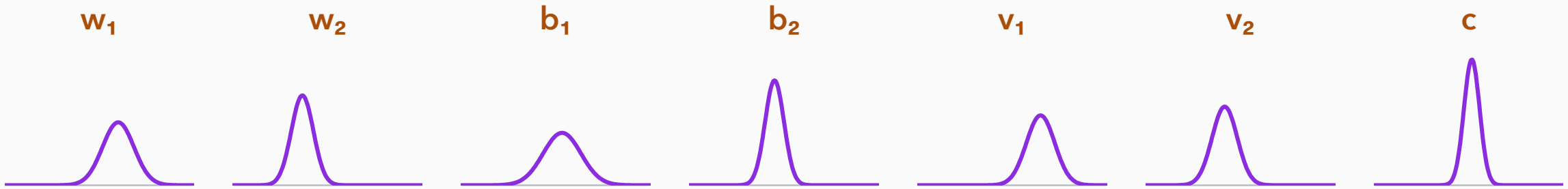
$$q_\psi(\theta) = \prod_{j=1}^{7} \mathcal{N}(\theta_j;\, \mu_j,\, \sigma_j^2), \qquad \psi = (\mu_1, \ldots, \mu_7,\, \sigma_1, \ldots, \sigma_7)$$



One bell per parameter. Together they define q(θ), and drawing θ from q is easy.

Variational inference: choose q, make it close to the posterior

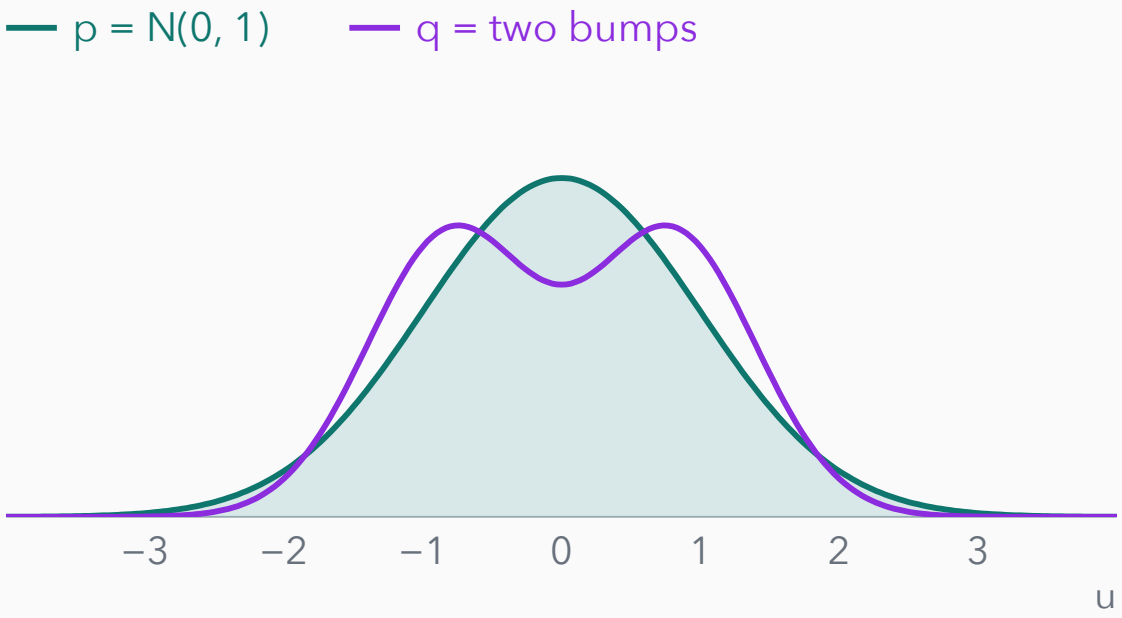
$$q_\psi(\theta) = \prod_{j=1}^{7} \mathcal{N}(\theta_j;\, \mu_j,\, \sigma_j^2), \qquad \psi = (\mu_1, \ldots, \mu_7,\, \sigma_1, \ldots, \sigma_7)$$



$$\psi^* = \arg\min_{\psi} \; \text{distance}\big(q_\psi(\theta),\; p(\theta \mid \mathcal{D})\big)$$

> Tune ψ until q is as close as possible to the posterior. Two questions: what is "close", and how do we tune without the posterior?

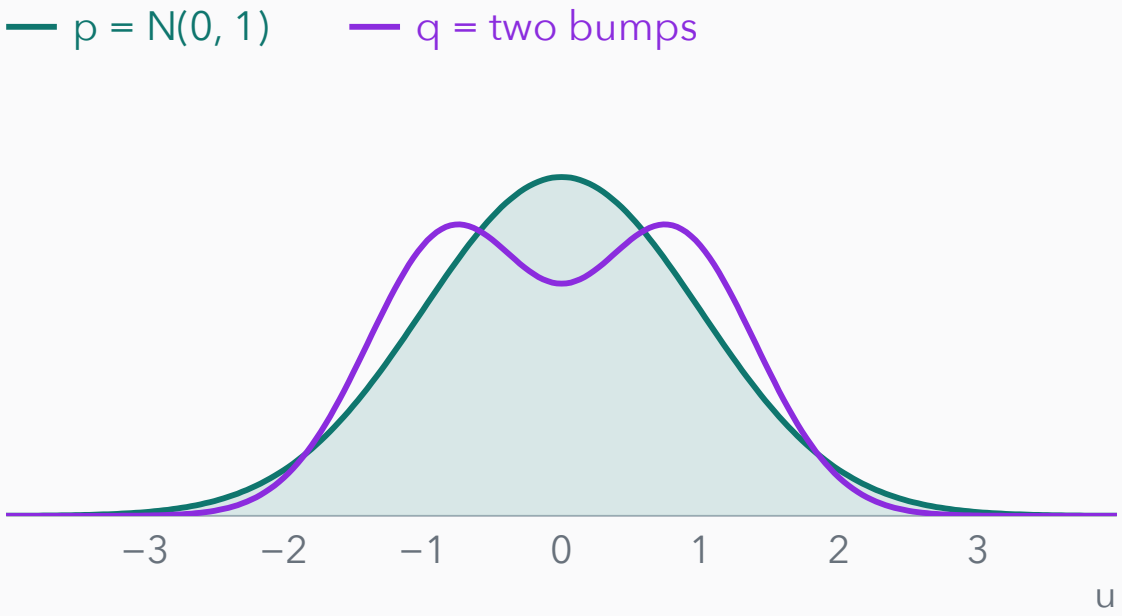
How far apart are two distributions?



	p	q
mean	0	0
variance	1	1

> Comparing means and variances is a start.

How far apart are two distributions?

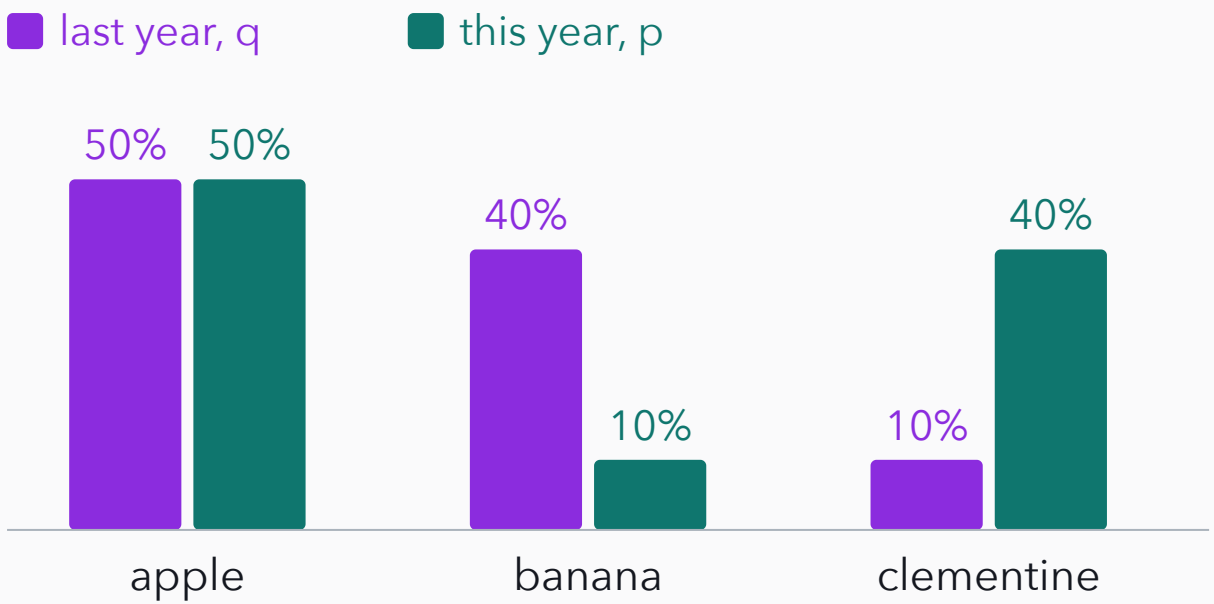


	p	q
mean	0	0
variance	1	1

Same mean, same variance, different distributions. We need one number that compares the whole shapes.

> But here both match, and the shapes still differ. We want a number that compares the distributions everywhere.

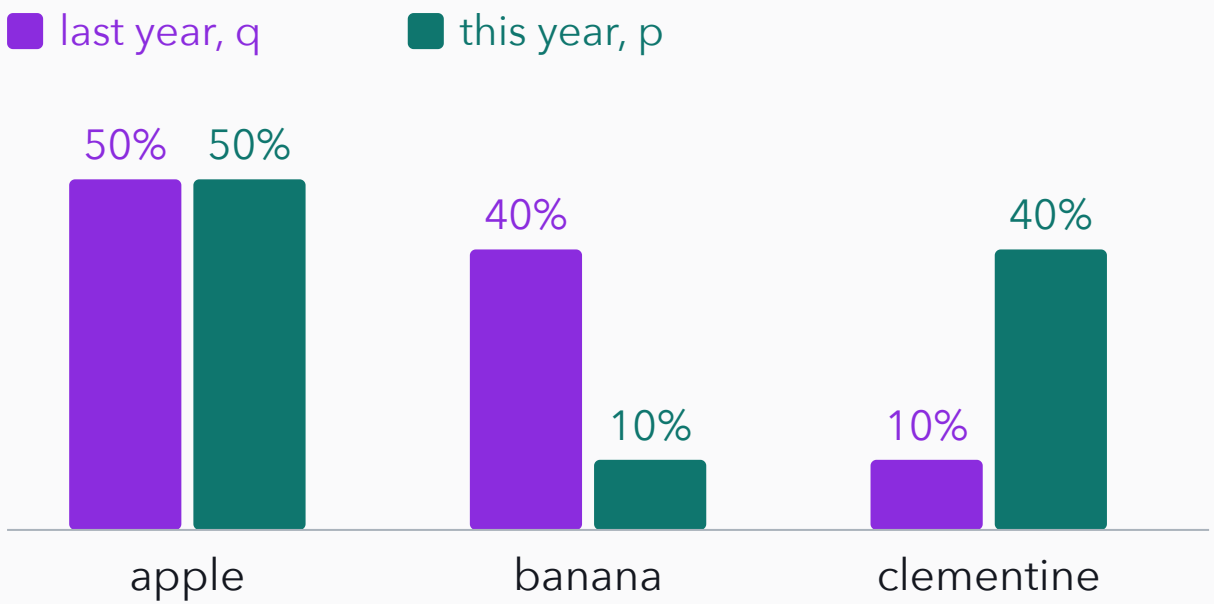
Two surveys: how different are they?



Favourite fruit in a class, last year (q) and this year (p).

> We want one number for how much the class changed.

Two surveys: how different are they?



Favourite fruit in a class, last year (q) and this year (p).

Bananas fell from 40% to 10%.
Clementines rose from 10% to 40%.

A fourfold fall and a fourfold rise. Should they count the same?

Ratios say “how many times”; logs make up and down cancel

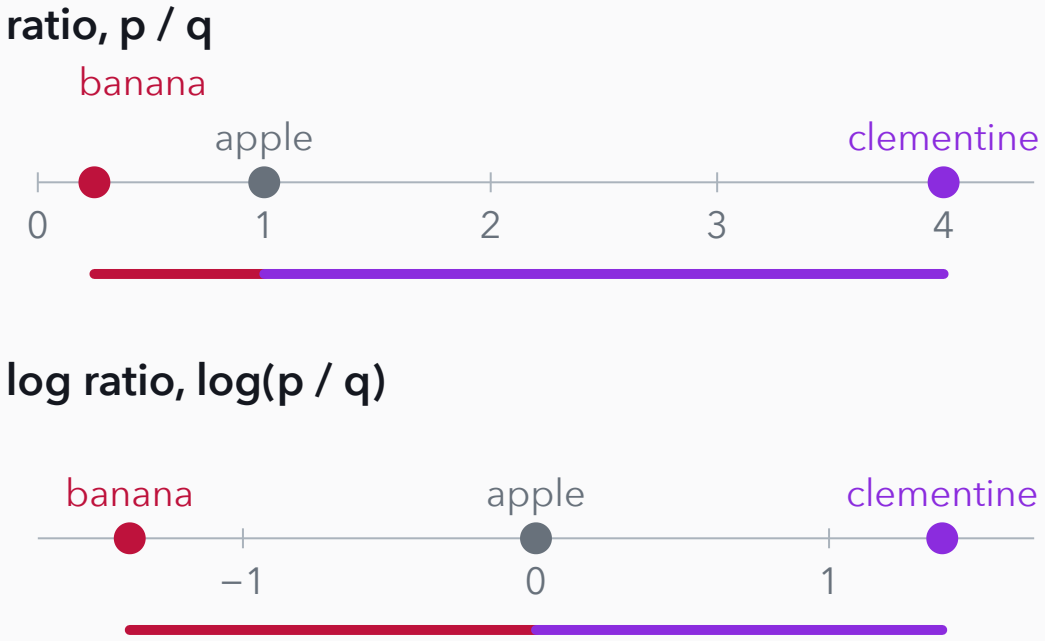
fruit	p	q	p/q	$\log(p/q)$
apple	0.5	0.5	1	
banana	0.1	0.4	¼	
clementine	0.4	0.1	4	



Ratios p/q: 1, ¼ and 4. But on the ratio scale, ×4 lands 3 above 1 and ÷4 only 0.75 below.

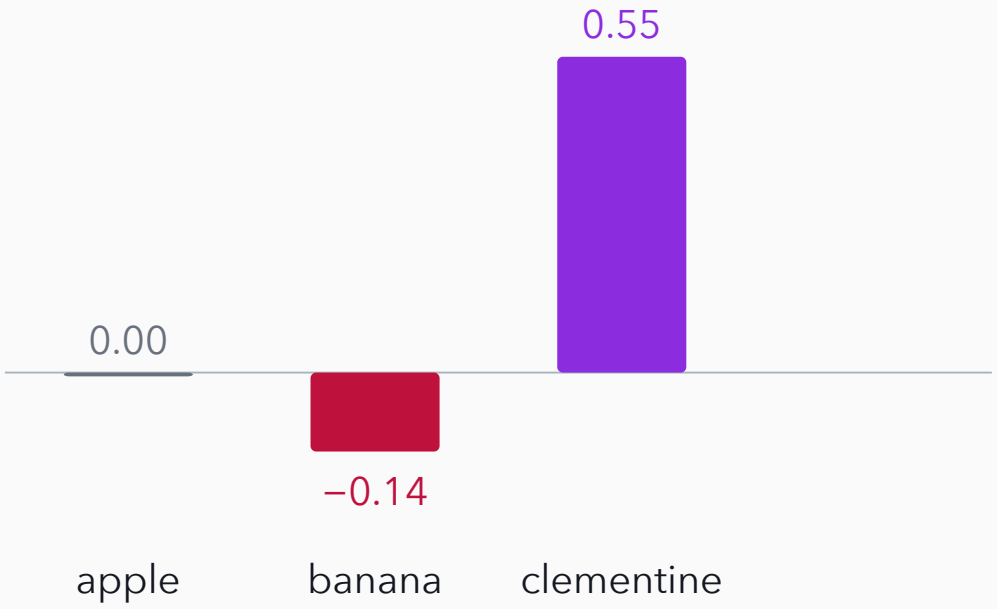
Ratios say “how many times”; logs make up and down cancel

fruit	p	q	p/q	$\log(p/q)$
apple	0.5	0.5	1	0.00
banana	0.1	0.4	¼	−1.39
clementine	0.4	0.1	4	1.39



Take logs: log 4 = +1.39 and log ¼ = −1.39. Equal and opposite.

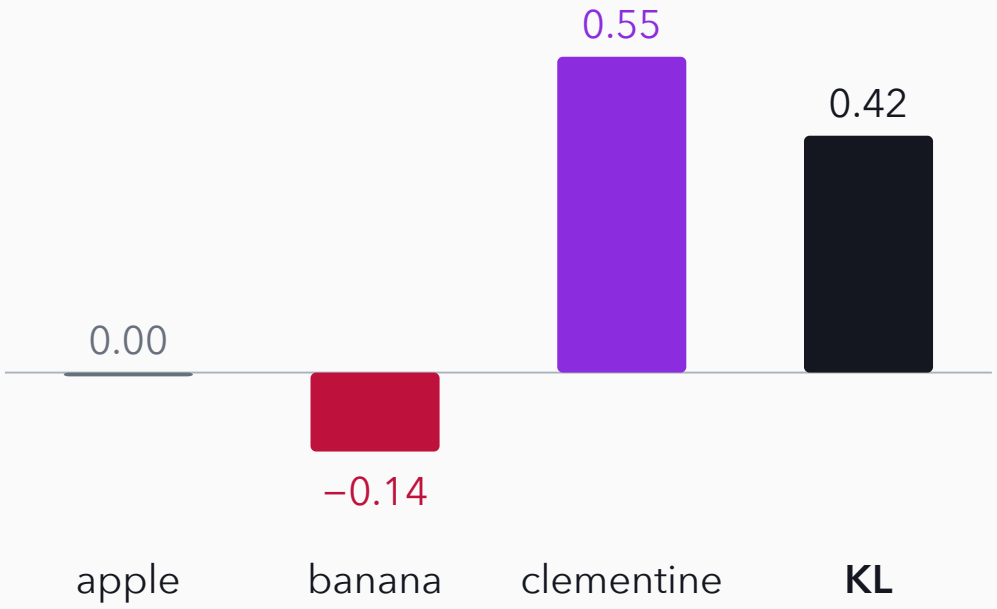
Weight by how common each fruit is now: that is KL



$$\mathrm{KL}(p \,\|\, q) = \sum_x p(x) \, \log \frac{p(x)}{q(x)}$$

> Average the log ratios, weighting each fruit by p(x), its share this year.

Weight by how common each fruit is now: that is KL



$$\mathrm{KL}(p \,\|\, q) = \sum_x p(x) \, \log \frac{p(x)}{q(x)}$$

$$= 0.5(0) + 0.1(-1.39) + 0.4(1.39)$$

$$\approx 0.42 \text{ nats}$$

> The rare banana barely counts; the common clementine dominates. Total 0.42 nats: the Kullback–Leibler divergence.

KL is the extra message length of using the wrong code

weather	p	q	code built for p	code built for q
sunny	0.50	0.25	`0`	`10`
cloudy	0.25	0.50	`10`	`0`
rainy	0.25	0.25	`11`	`11`

$$\text{average length under } p: \quad \text{code for } p = 1.50 \text{ bits}, \qquad \text{code for } q = 1.75 \text{ bits}$$

> Send each day's weather in bits. A code built for the true frequencies p needs 1.50 bits a day.

KL is the extra message length of using the wrong code

weather	p	q	code built for p	code built for q
sunny	0.50	0.25	`0`	`10`
cloudy	0.25	0.50	`10`	`0`
rainy	0.25	0.25	`11`	`11`

$$\text{average length under } p: \quad \text{code for } p = 1.50 \text{ bits}, \qquad \text{code for } q = 1.75 \text{ bits}$$

$$1.75 - 1.50 = 0.25 \text{ bits} = \mathrm{KL}(p \| q) \text{ with } \log_2$$

> A code built for the wrong frequencies q needs 1.75. The extra 0.25 bits a day are exactly KL(p‖q), in base 2.

You have minimised KL before: in every cross-entropy loss

$$\underbrace{-\sum_x p(x)\log q(x)}_{\text{cross-entropy } H(p,q)} = \underbrace{-\sum_x p(x)\log p(x)}_{\text{entropy } H(p)} + \mathrm{KL}(p\|q)$$

Cross-entropy splits into entropy plus KL: the 1.75 bits were 1.50 + 0.25.

You have minimised KL before: in every cross-entropy loss

$$\underbrace{-\sum_x p(x)\log q(x)}_{\text{cross-entropy } H(p,q)} = \underbrace{-\sum_x p(x)\log p(x)}_{\text{entropy } H(p)} + \mathrm{KL}(p\|q)$$

$$\text{classifier:}\quad \min_\theta H(y,\, q_\theta) \iff \min_\theta \mathrm{KL}(y \,\|\, q_\theta)$$

A classifier's cross-entropy loss: the labels' entropy is fixed, so training minimises a KL.

You have minimised KL before: in every cross-entropy loss

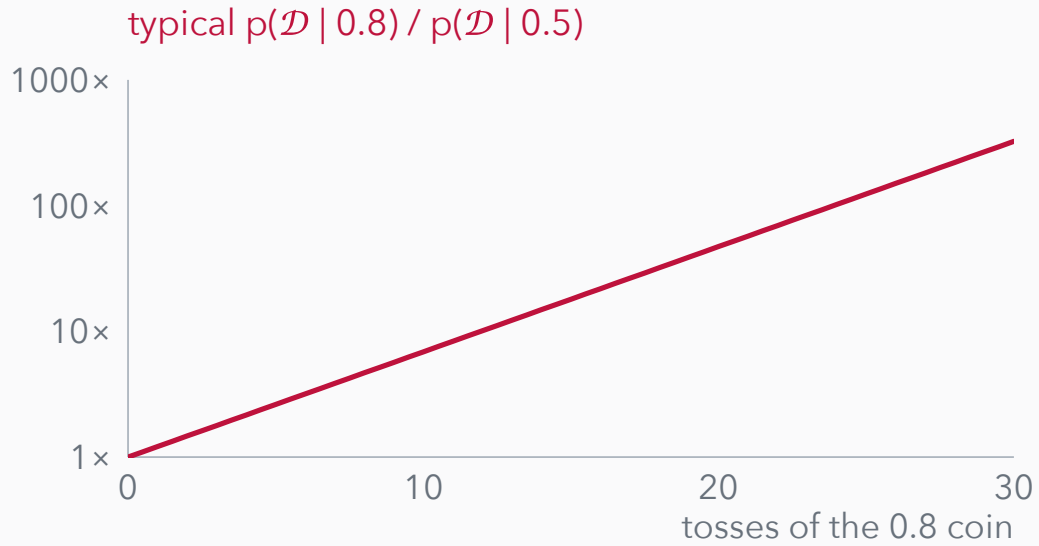
$$\underbrace{-\sum_x p(x)\log q(x)}_{\text{cross-entropy } H(p,q)} = \underbrace{-\sum_x p(x)\log p(x)}_{\text{entropy } H(p)} + \mathrm{KL}(p\|q)$$

$$\text{classifier:} \quad \min_\theta H(y,\, q_\theta) \iff \min_\theta \mathrm{KL}(y \,\|\, q_\theta)$$

$$\text{maximum likelihood:} \quad -\tfrac{1}{N}\textstyle\sum_i \log p_\theta(x_i) \;\approx\; H(p_{\text{data}}) + \mathrm{KL}(p_{\text{data}} \,\|\, p_\theta)$$

> Maximum likelihood minimises KL(data ‖ model). Variational inference is the third use.

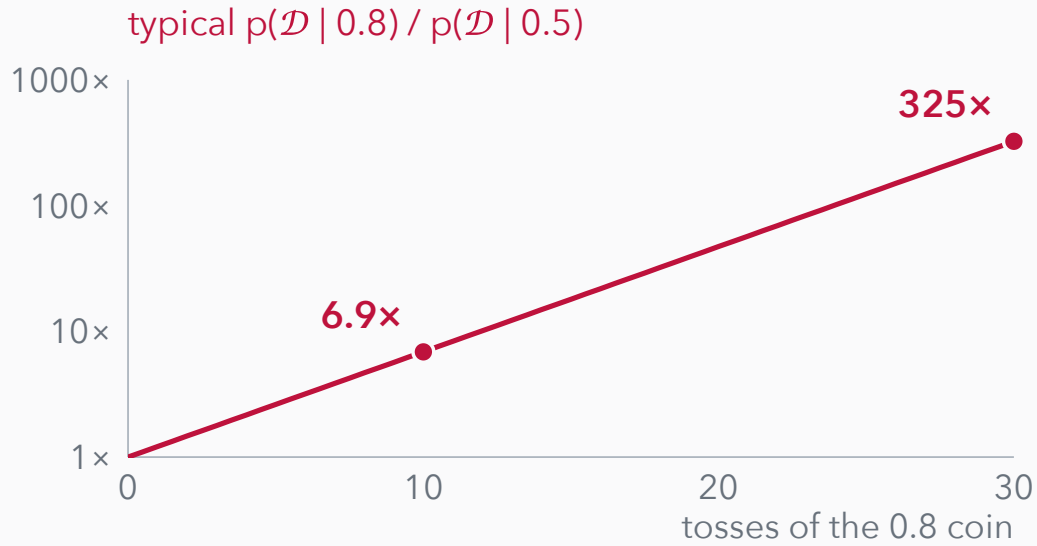
A KL value is how fast evidence piles up



$$\mathrm{KL}\big(\mathrm{Bern}(0.8) \,\|\, \mathrm{Bern}(0.5)\big) \approx 0.19 \text{ nats}$$

> A coin lands heads 80% of the time. Each toss favours “0.8” over “fair” by 0.19 nats on average. After 30 tosses, by what factor?

A KL value is how fast evidence piles up



$$\mathrm{KL}\big(\mathrm{Bern}(0.8) \,\|\, \mathrm{Bern}(0.5)\big) \approx 0.19 \text{ nats}$$

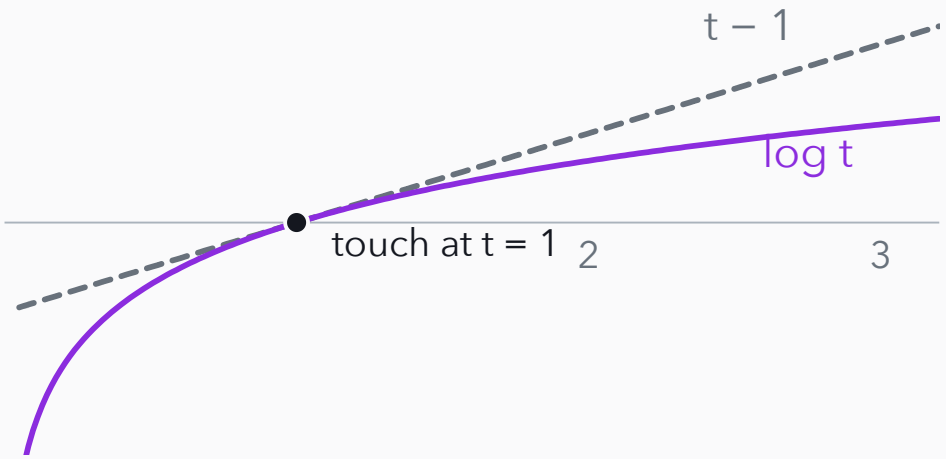
$$\frac{p(\mathcal{D} \mid 0.8)}{p(\mathcal{D} \mid 0.5)} \approx e^{0.19\,n}$$

The reverse direction, KL(0.5‖0.8) = 0.22, is a different number.

About e^(30 × 0.19) ≈ 325. KL is the rate at which evidence piles up: 6.9× after 10 tosses, 325× after 30.

For densities, sums become integrals, and KL ≥ 0

$$\mathrm{KL}(q \,\|\, p) = \int q(u) \, \log \frac{q(u)}{p(u)} \, du$$



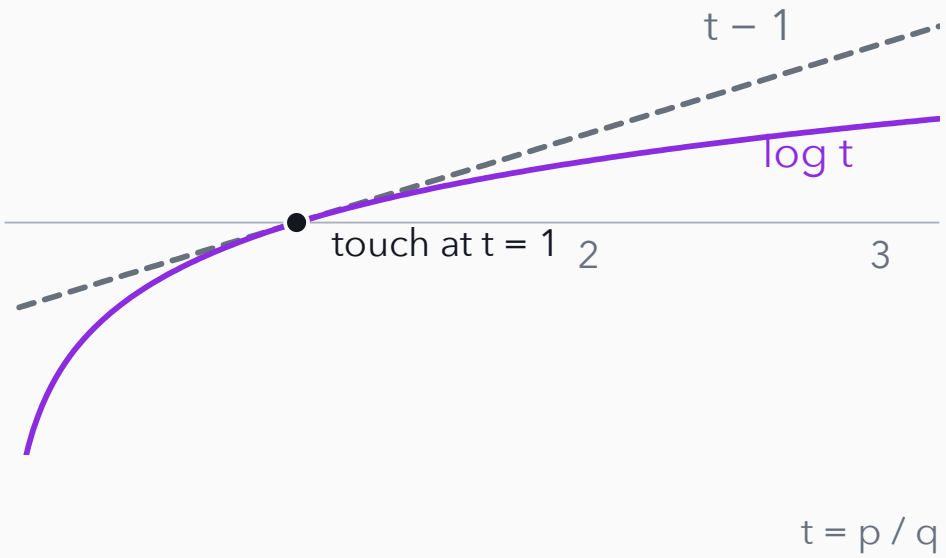
For a continuous unknown u the sum becomes an integral. Same recipe: weight the log ratio by the first argument.

For densities, sums become integrals, and KL ≥ 0

$$\mathrm{KL}(q \,\|\, p) = \int q(u)\, \log \frac{q(u)}{p(u)}\, du$$

$$-\mathrm{KL}(q\|p) = \mathbb{E}_q\left[\log \frac{p}{q}\right] \le \mathbb{E}_q\left[\frac{p}{q} - 1\right]$$

$$= \int p\, du - 1 = 0$$



log t never rises above its tangent t − 1. Inside the average that gives KL ≥ 0, with 0 only when q = p.

Closed forms for simple distributions

distributions	$\mathrm{KL}(p \,\|\, q)$	example
Bernoulli(a), Bernoulli(b)	$a \log \frac{a}{b} + (1-a) \log \frac{1-a}{1-b}$	coin, 0.8 vs 0.5: 0.19
categorical	$\sum_k p_k \log \frac{p_k}{q_k}$	fruit surveys: 0.42
Gaussians	$\log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$	derived next
diagonal Gaussians	sum of one-dimensional terms	our q against the prior

> For simple pairs, the weighted average of log ratios has a closed form. These are the ones we will use.

Closed forms for simple distributions

distributions	$\mathrm{KL}(p \,\|\, q)$	example
Bernoulli(a), Bernoulli(b)	$a \log \frac{a}{b} + (1-a) \log \frac{1-a}{1-b}$	coin, 0.8 vs 0.5: 0.19
categorical	$\sum_k p_k \log \frac{p_k}{q_k}$	fruit surveys: 0.42
Gaussians	$\log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$	derived next
diagonal Gaussians	sum of one-dimensional terms	our q against the prior

A mixture, or a posterior, has no closed form. Then we estimate the average by sampling.

> Anything else, such as a mixture or a posterior, has no formula; then we sample.

Deriving the Gaussian formula

$$q = \mathcal{N}(\mu, \sigma^2), \quad p = \mathcal{N}(m, s^2): \qquad \log \frac{q(u)}{p(u)} = \log \frac{s}{\sigma} + \frac{(u-m)^2}{2s^2} - \frac{(u-\mu)^2}{2\sigma^2}$$

Write the log ratio of the two densities. The √(2π) terms cancel.

Deriving the Gaussian formula

$$q = \mathcal{N}(\mu, \sigma^2), \quad p = \mathcal{N}(m, s^2): \qquad \log \frac{q(u)}{p(u)} = \log \frac{s}{\sigma} + \frac{(u-m)^2}{2s^2} - \frac{(u-\mu)^2}{2\sigma^2}$$

$$\mathbb{E}_q[(u-m)^2] = \sigma^2 + (\mu - m)^2, \qquad \mathbb{E}_q[(u-\mu)^2] = \sigma^2$$

> Average it under q. Two moments are all we need.

Deriving the Gaussian formula

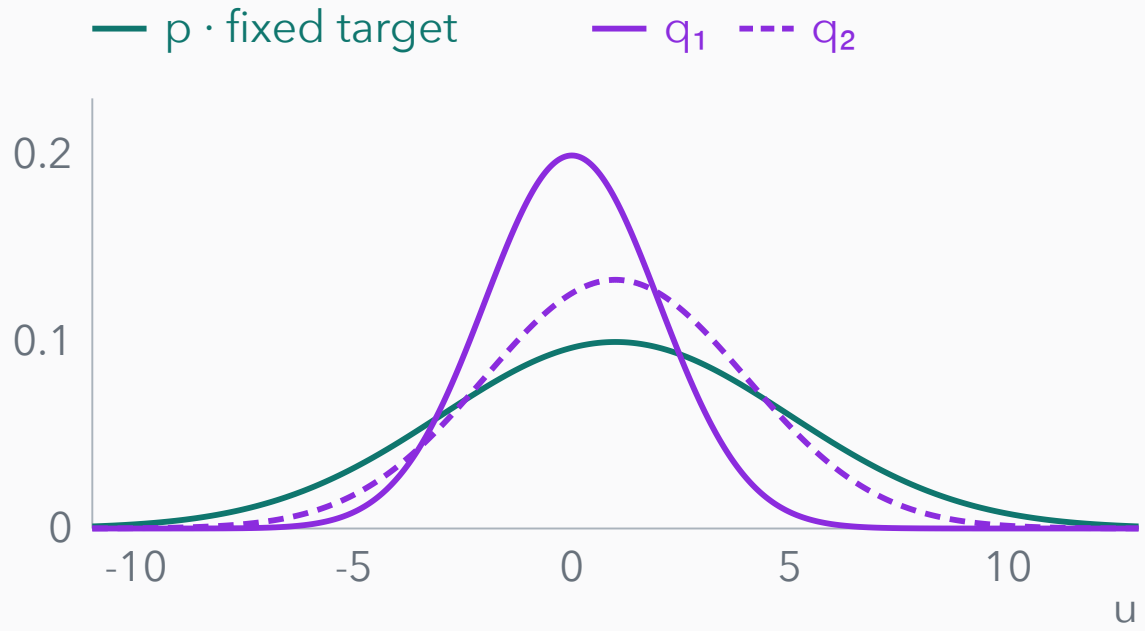
$$q = \mathcal{N}(\mu, \sigma^2), \quad p = \mathcal{N}(m, s^2): \qquad \log \frac{q(u)}{p(u)} = \log \frac{s}{\sigma} + \frac{(u-m)^2}{2s^2} - \frac{(u-\mu)^2}{2\sigma^2}$$

$$\mathbb{E}_q[(u-m)^2] = \sigma^2 + (\mu - m)^2, \qquad \mathbb{E}_q[(u-\mu)^2] = \sigma^2$$

$$\mathrm{KL}(q\|p) = \log \frac{s}{\sigma} + \frac{\sigma^2 + (\mu - m)^2}{2s^2} - \frac{1}{2}$$

The result: a width term, a squared shift of the centres, and −½.

Use the formula: which Gaussian is closer to p?

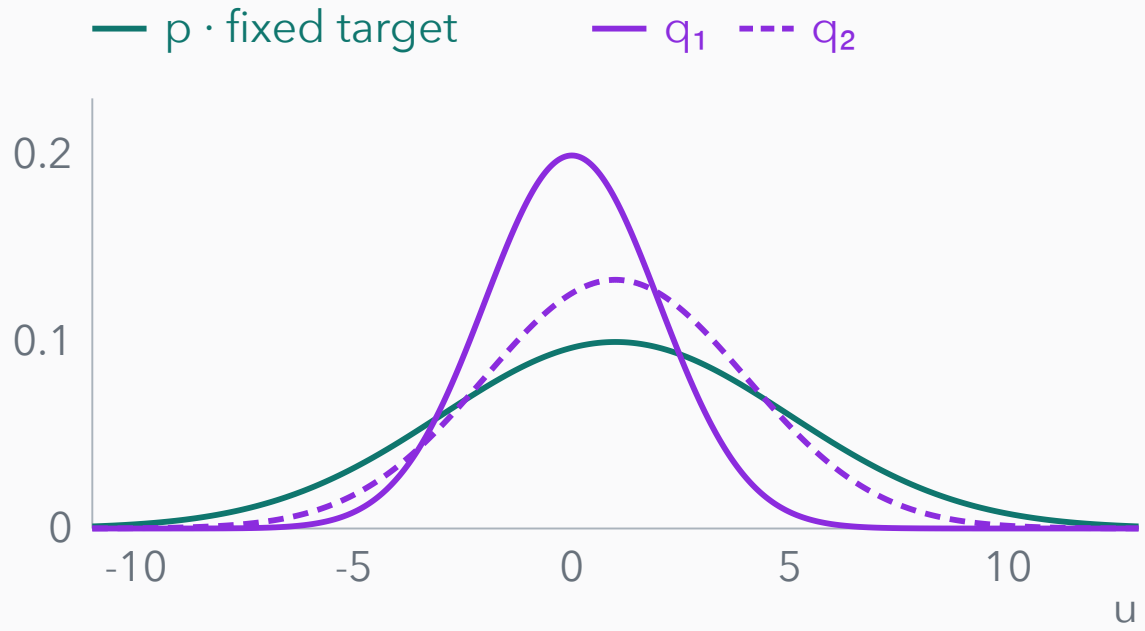


$$p = \mathcal{N}(1, 4^2)$$

	substitute	KL
$q_1 = \mathcal{N}(0, 2^2)$	$\log 2 + \frac{4+1}{32} - \frac{1}{2}$	
$q_2 = \mathcal{N}(1, 3^2)$	$\log \frac{4}{3} + \frac{9+0}{32} - \frac{1}{2}$	

The PML vote: which is closer to p, q_1 or q_2? Substitute into the formula.

Use the formula: which Gaussian is closer to p?

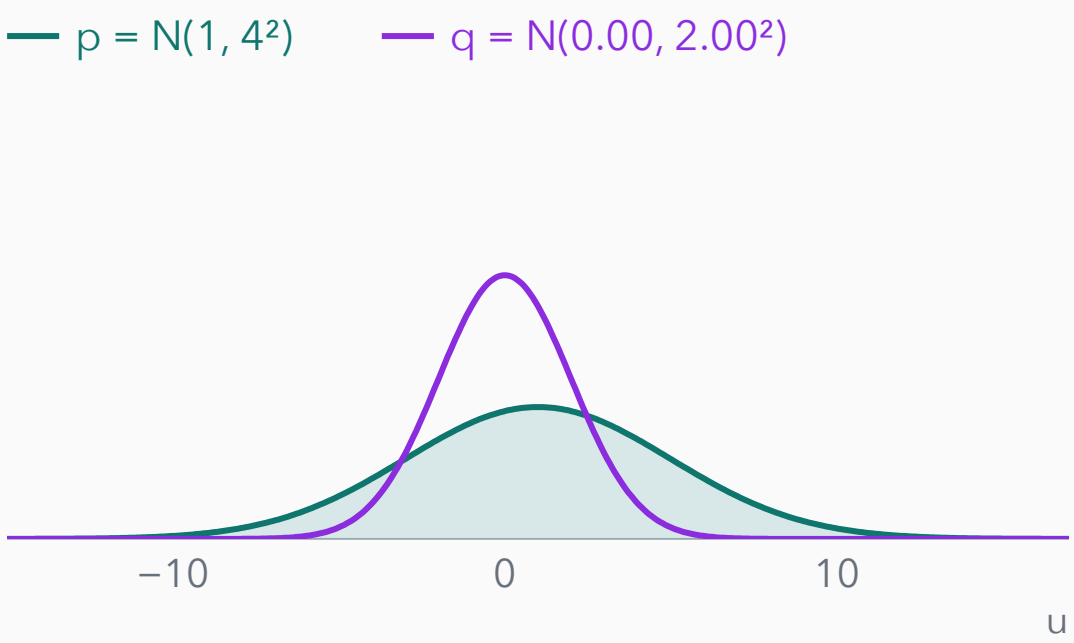


$$p = \mathcal{N}(1, 4^2)$$

	substitute	KL
$q_1 = \mathcal{N}(0, 2^2)$	$\log 2 + \frac{4+1}{32} - \frac{1}{2}$	0.35
$q_2 = \mathcal{N}(1, 3^2)$	$\log \frac{4}{3} + \frac{9+0}{32} - \frac{1}{2}$	0.07

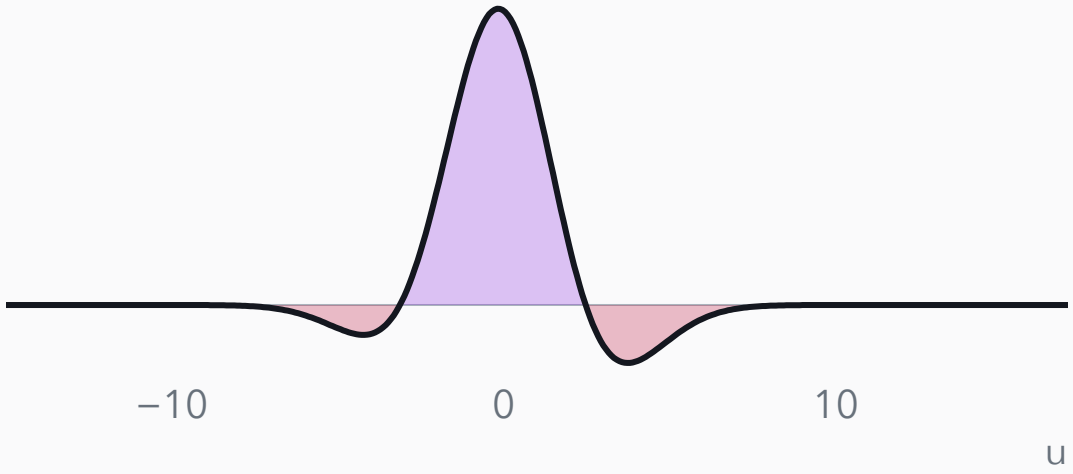
KL(q_1‖p) = 0.35 nats and KL(q_2‖p) = 0.07. The formula settles the vote.

KL is an area you can watch



net area = KL(qllp) = 0.35

q log(q/p): purple adds, red subtracts



Purple area adds and red area subtracts; the net area is the KL. Drag q, or flip the direction.

No formula needed: estimate KL by sampling

$$\mathrm{KL}(q\|p) = \mathbb{E}_{u\sim q}\big[\log q(u) - \log p(u)\big] \approx \frac{1}{S}\sum_{i=1}^{S}\big[\log q(u_i) - \log p(u_i)\big]$$

```
u = q.sample((S,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
est = (log_q - log_p).mean()
exact = kl_divergence(q, p)
```

Draw u from q, score each draw with both log densities, and average. No formula needed.

No formula needed: estimate KL by sampling

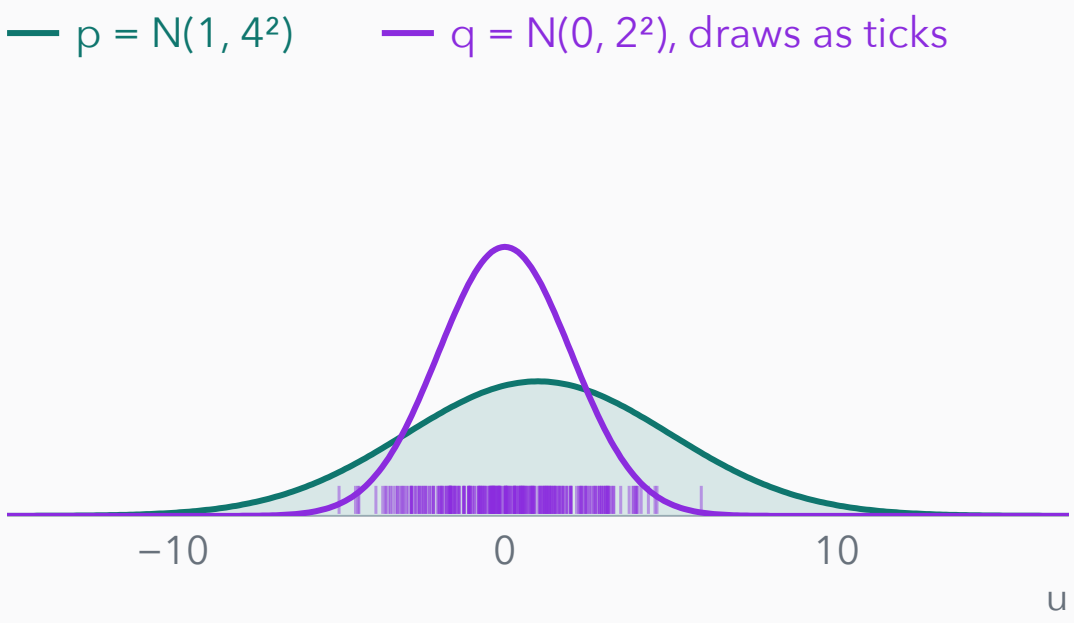
$$\mathrm{KL}(q\|p) = \mathbb{E}_{u\sim q}\big[\log q(u) - \log p(u)\big] \approx \frac{1}{S}\sum_{i=1}^{S}\big[\log q(u_i) - \log p(u_i)\big]$$

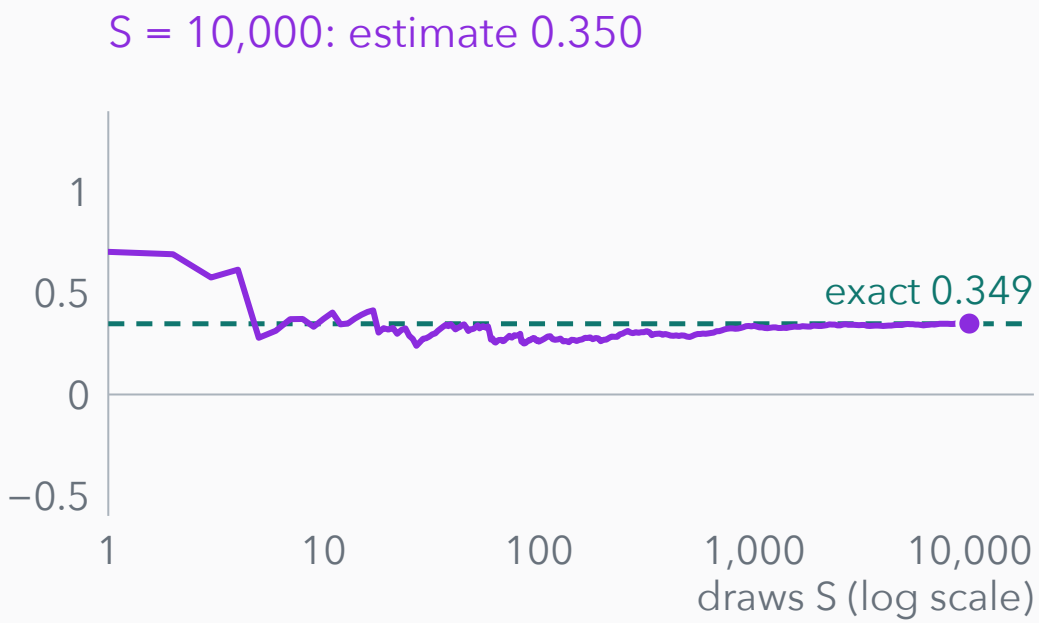
```
u = q.sample((S,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
est = (log_q - log_p).mean()
exact = kl_divergence(q, p)
```

	$KL(q_2\|\|p)$
exact formula	0.069
estimate, S = 10,000	0.067

Checked on a pair that has a formula: 0.067 from 10,000 draws, against the exact 0.069.

Watch the estimate settle

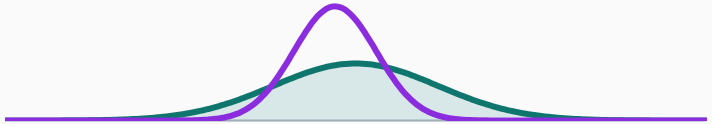




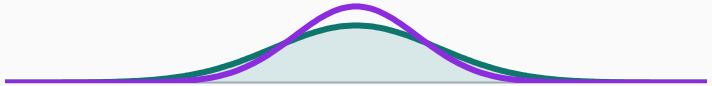
Add draws from q and watch the running average of log q − log p settle on the exact 0.349.

Six candidates: which q is closest to p?

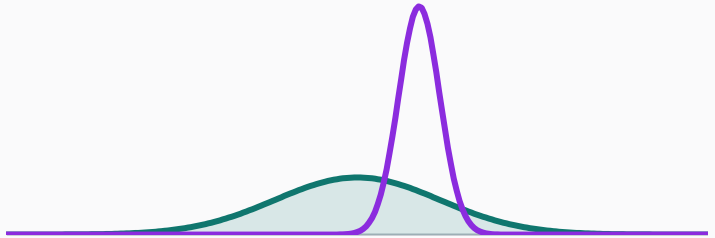
$q = N(0, 2^2)$



$q = N(1, 3^2)$



$q = N(4, 1^2)$



$q = N(-3, 4^2)$



$q = N(1, 8^2)$



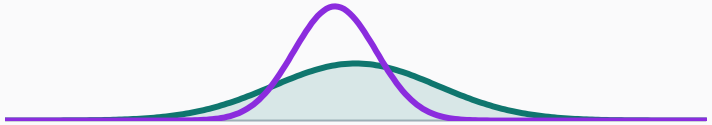
$q = N(2, 5^2)$



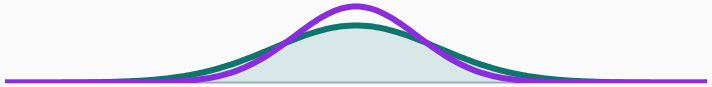
Six Gaussians against the same target $p = N(1, 4^2)$. Vote: which is closest?

Six candidates: which q is closest to p?

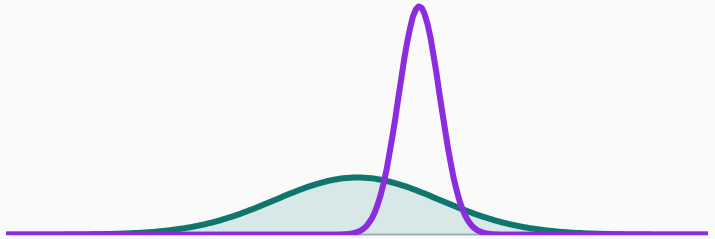
$q = N(0, 2^2)$ KL 0.35



$q = N(1, 3^2)$ **KL 0.07**



$q = N(4, 1^2)$ KL 1.20



$q = N(-3, 4^2)$ KL 0.50



$q = N(1, 8^2)$ KL 0.81

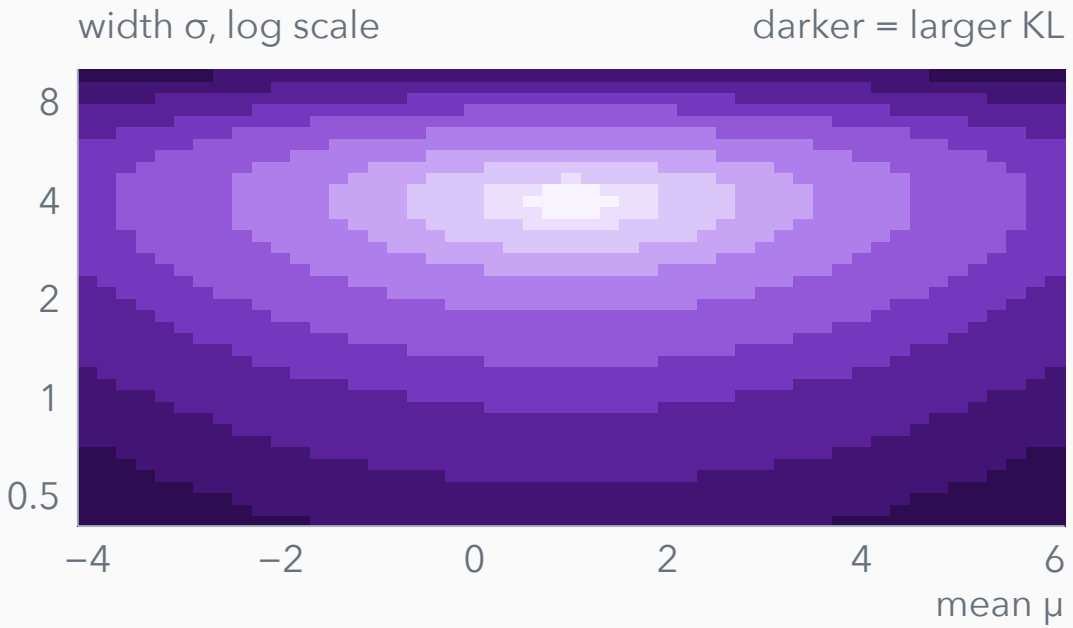


$q = N(2, 5^2)$ KL 0.09



Smallest KL: $N(1, 3^2)$ at 0.07, then $N(2, 5^2)$ at 0.09. Could another q do better?

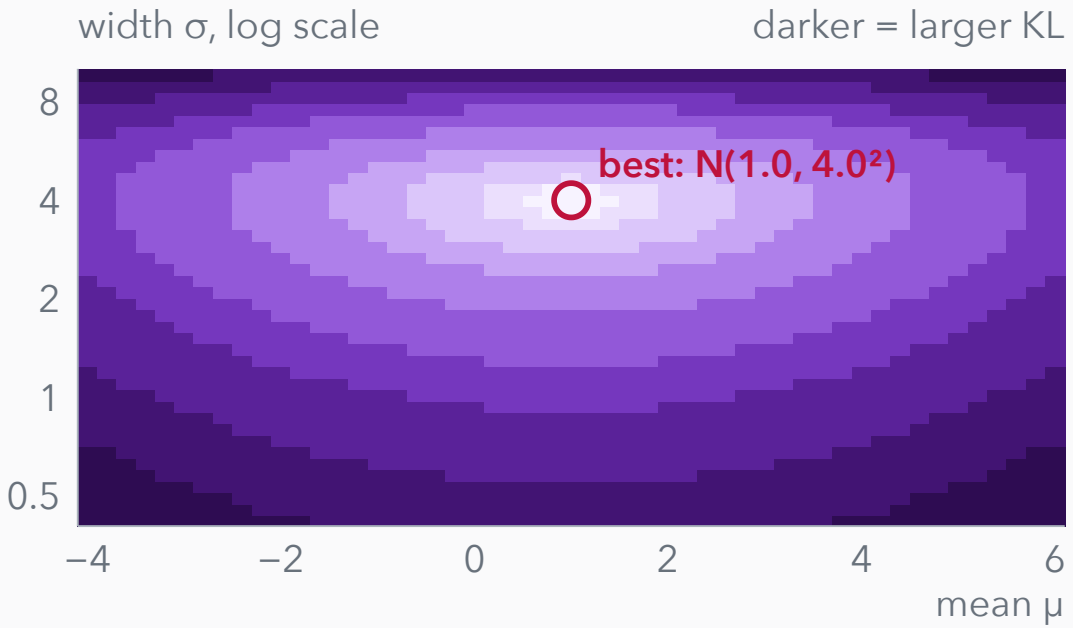
Search a grid of means and widths



Evaluate $\mathrm{KL}(q \| p)$ at every point of a 51 × 40 grid of means and widths: 2,040 evaluations of the formula.

> Try every (μ, σ) on a grid and colour each by its KL. Darker means further from p.

Search a grid of means and widths

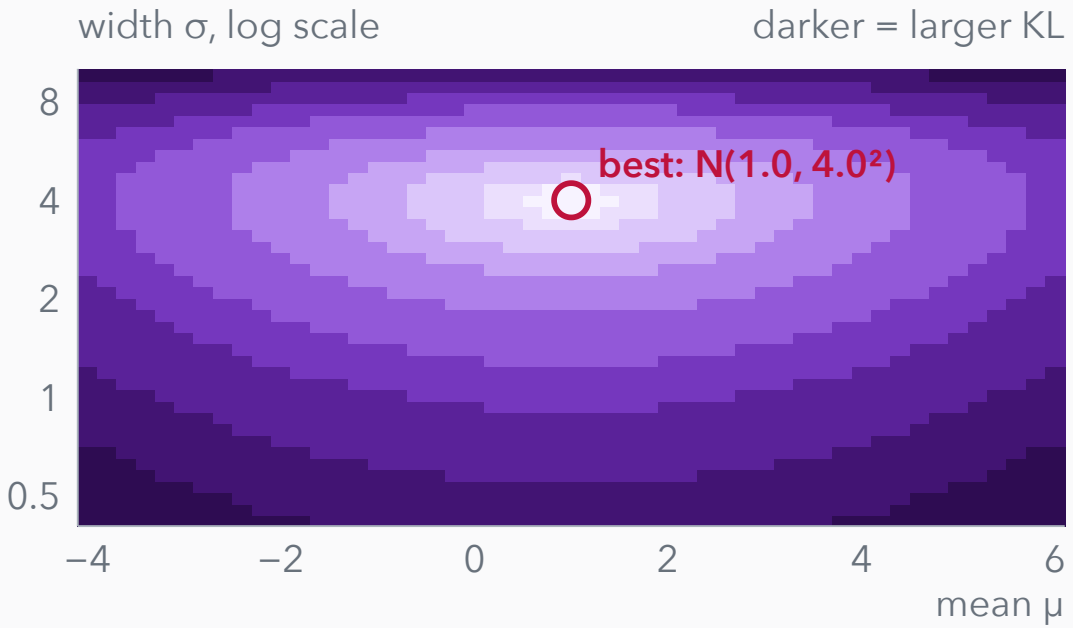


Evaluate $\mathrm{KL}(q\|p)$ at every point of a 51 × 40 grid of means and widths: 2,040 evaluations of the formula.

Best grid point: q = p. But our network's q has 14 numbers; a grid with 40 values each needs $40^{14} \approx 10^{22}$ evaluations.

> The best grid point is N(1, 4²), which is p. For the network's fourteen numbers, a grid is hopeless.

Instead of a grid, walk downhill

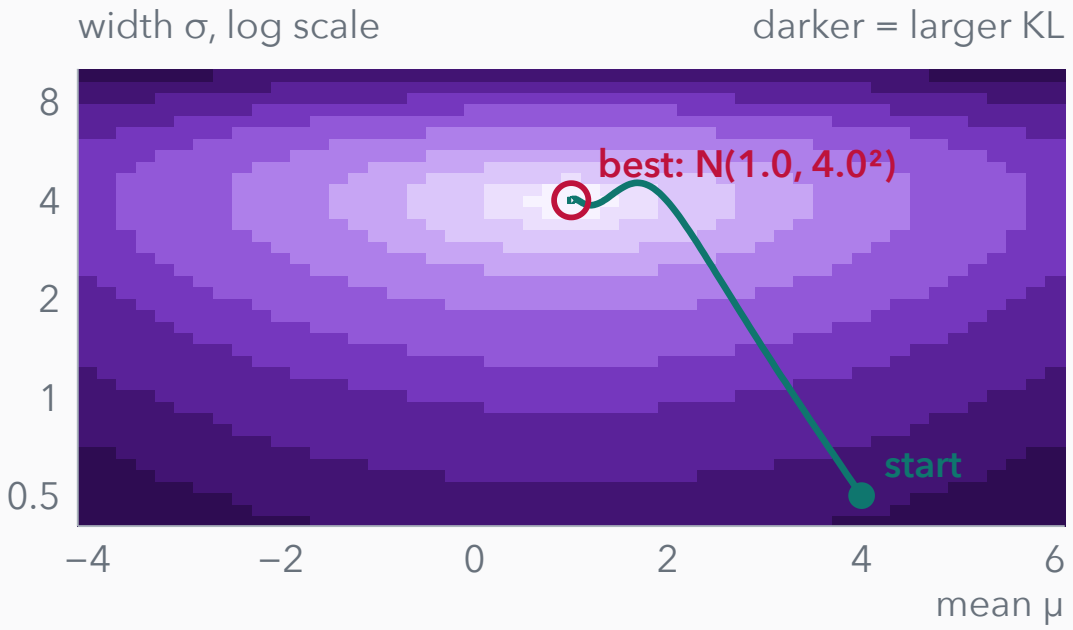


$$\psi^* = \arg\min_{\psi} \mathrm{KL}(q_\psi \,\|\, p)$$

$$\psi \leftarrow \psi - \eta \, \nabla_\psi \mathrm{KL}(q_\psi \,\|\, p)$$

Cast the search as optimisation: follow the gradient of KL with respect to q's parameters.

Instead of a grid, walk downhill



$$\psi^* = \arg\min_{\psi} \mathrm{KL}(q_\psi \,\|\, p)$$

$$\psi \leftarrow \psi - \eta \, \nabla_\psi \mathrm{KL}(q_\psi \,\|\, p)$$

Gradient descent from N(4, 0.5²): the teal path. About a hundred steps, and the same recipe works for millions of parameters.

From N(4, 0.5²) the optimiser walks downhill to p. It needs only gradients, which autograd provides.

The same search in code

```
mu  = Parameter(tensor(4.0))
rho = Parameter(tensor(log(0.5)))
opt = Adam([mu, rho], lr=0.05)
for step in range(200):
    q = Normal(mu, rho.exp())
    loss = kl_divergence(q, p)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

$$\sigma = \exp(\rho) > 0 \ \text{ for every } \rho$$

Store ρ = log σ, so that σ = exp(ρ) stays positive whatever an update does.

The same search in code

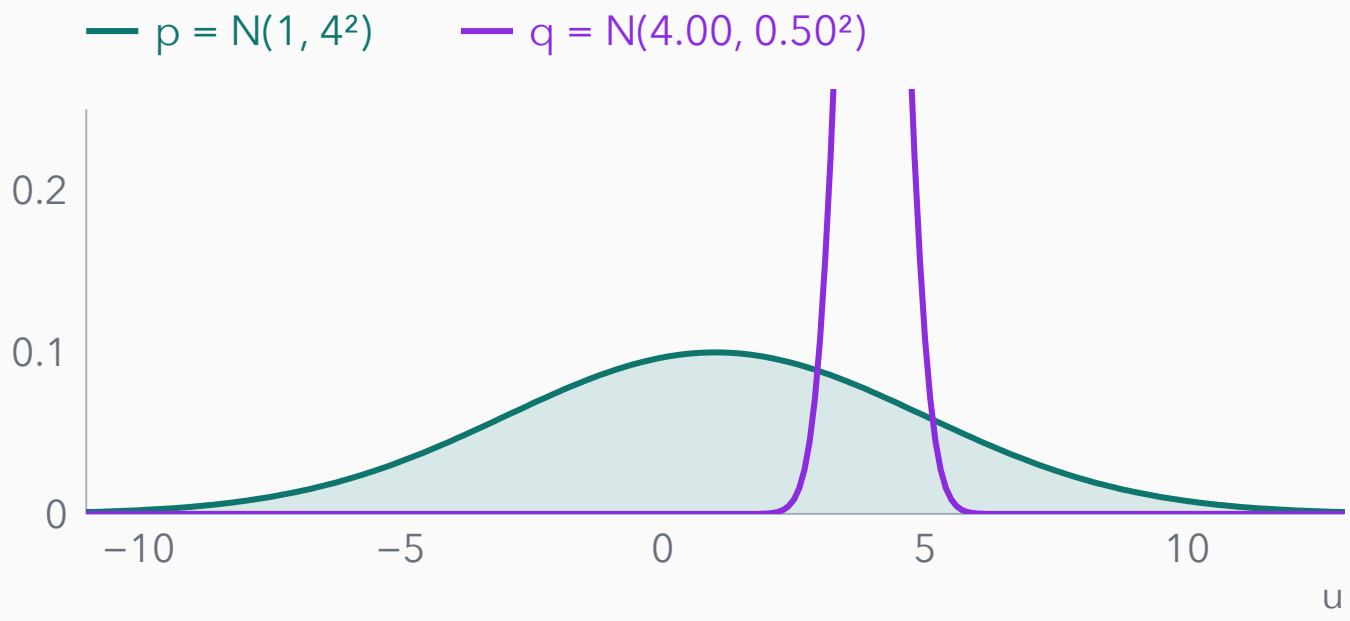
```
mu  = Parameter(tensor(4.0))
rho = Parameter(tensor(log(0.5)))
opt = Adam([mu, rho], lr=0.05)
for step in range(200):
    q = Normal(mu, rho.exp())
    loss = kl_divergence(q, p)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

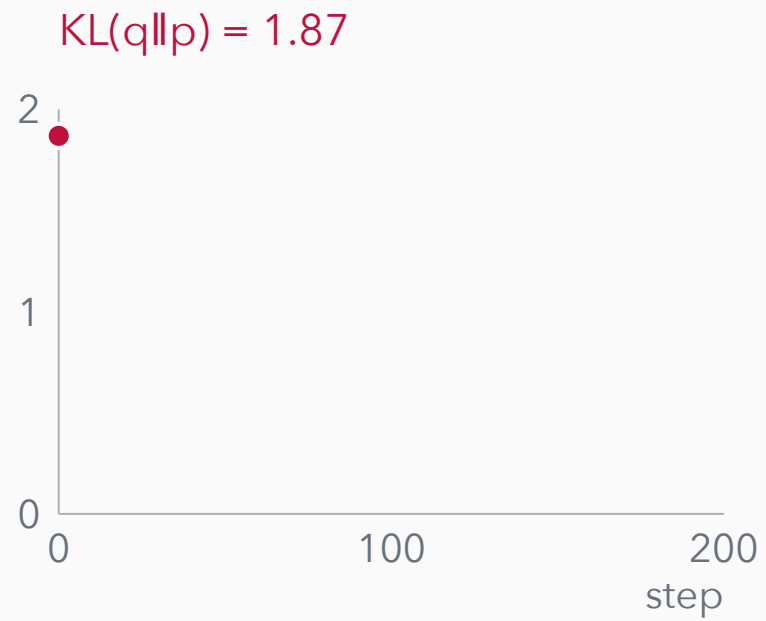
$$\sigma = \exp(\rho) > 0 \ \text{ for every } \rho$$

Rebuild q, score it, differentiate, update. The same loop fits our network in Part 5.

Rebuild q, score it, differentiate, update. The same five lines fit our network in Part 5.

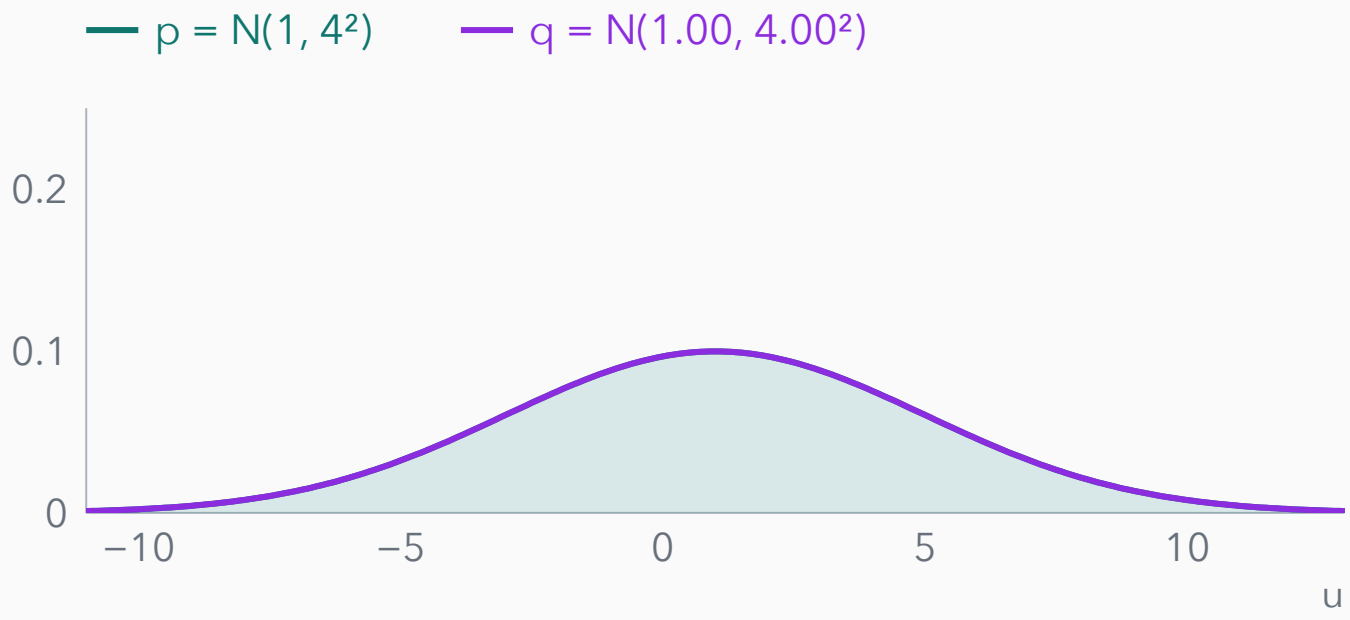
Press play: q slides onto p

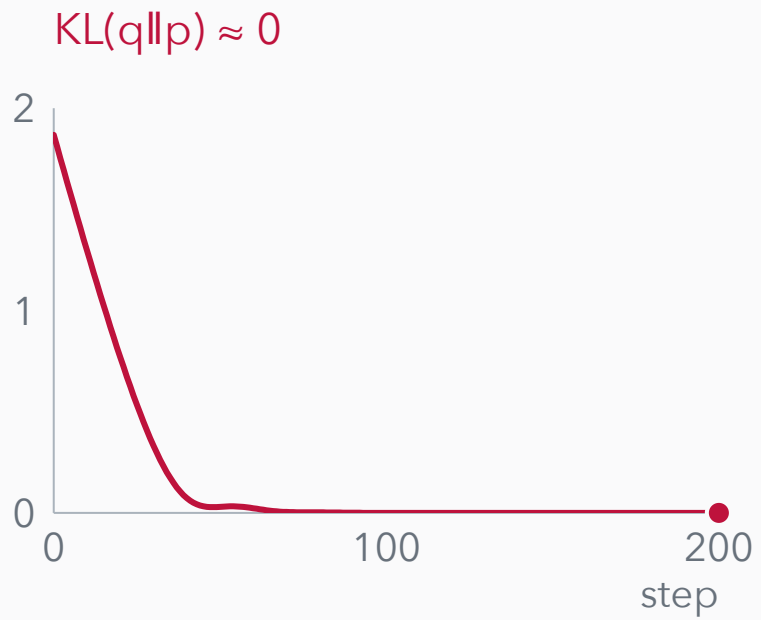




Start q far away, at N(4, 0.5²). Predict: which reaches the target first, the centre or the width?

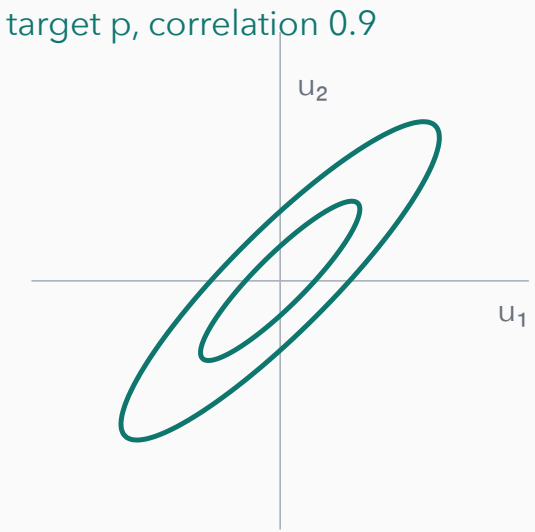
Press play: q slides onto p





> The width gets there first and overshoots a little; the centre slides in by step 100. Then KL ≈ 0.

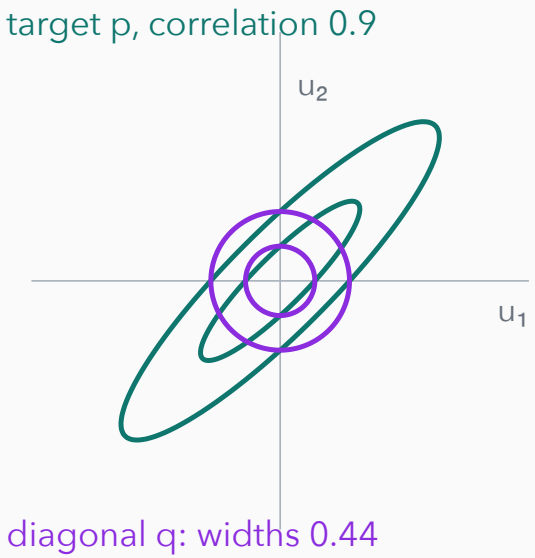
In 2-D, a q that cannot tilt shrinks instead



$$p = \mathcal{N}\left(0, \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}\right)$$

A correlated target: both coordinates have width 1, correlation 0.9. A diagonal q cannot tilt. Predict its widths.

In 2-D, a q that cannot tilt shrinks instead



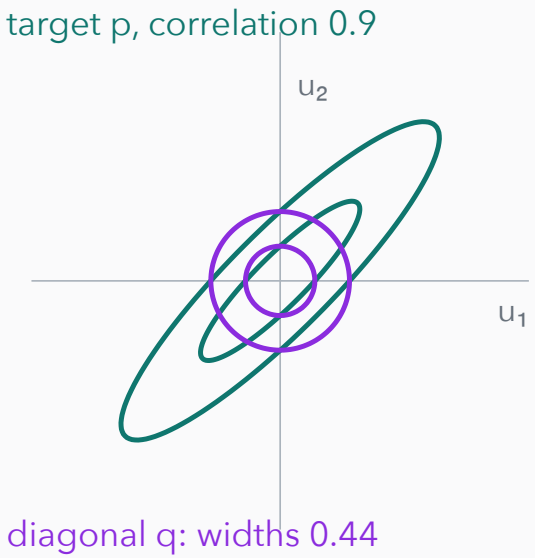
$$p = \mathcal{N}\left(0, \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}\right)$$

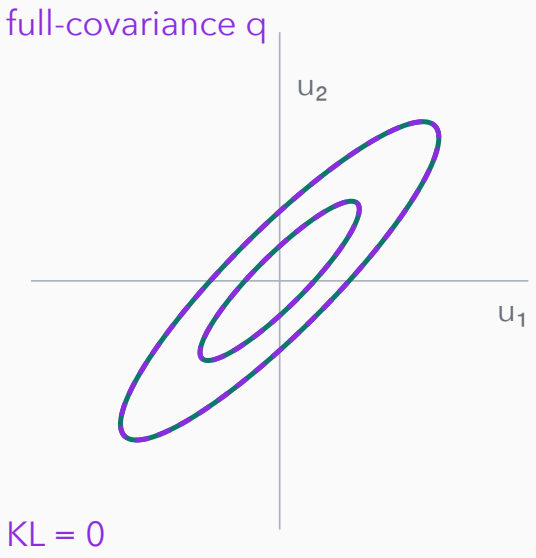
$$q = \mathcal{N}\left(0, \mathrm{diag}(s^2, s^2)\right)$$

$$s = 0.44, \quad \mathrm{KL} = 0.83$$

> It shrinks to fit inside the ellipse: widths 0.44, not 1. Mean-field VI underestimates uncertainty.

In 2-D, a q that cannot tilt shrinks instead





$$p = \mathcal{N}\left(0, \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}\right)$$

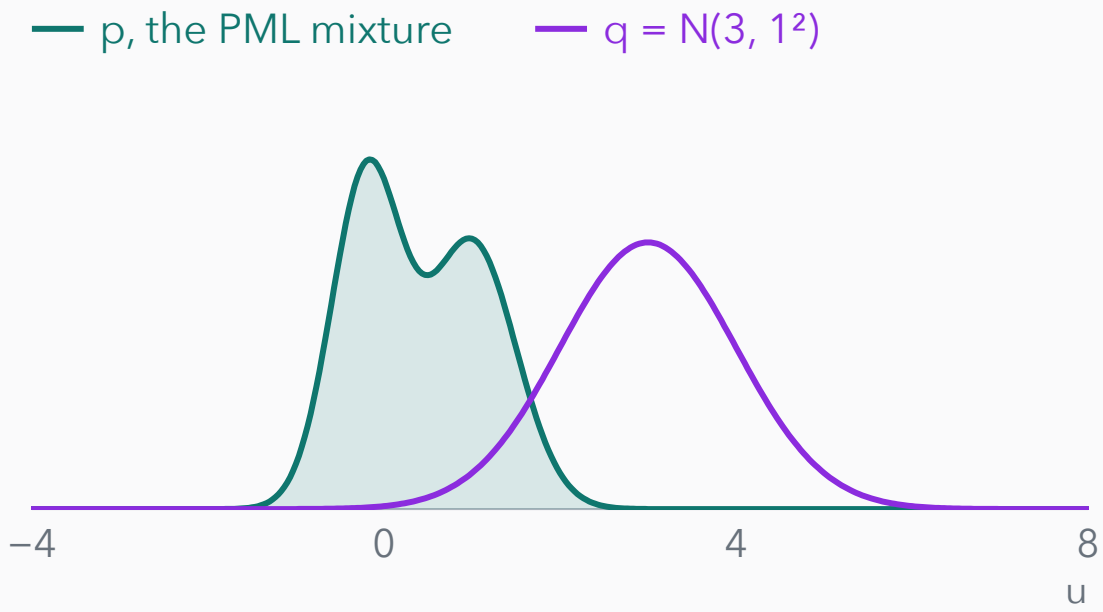
$$q = \mathcal{N}\left(0, \mathrm{diag}(s^2, s^2)\right)$$

$$s = 0.44, \quad \mathrm{KL} = 0.83$$

$$q = \mathcal{N}(0, LL^\top) : \ \mathrm{KL} = 0$$

> Let q tilt with a full covariance, Σ = LLᵀ, and the KL drops to 0.

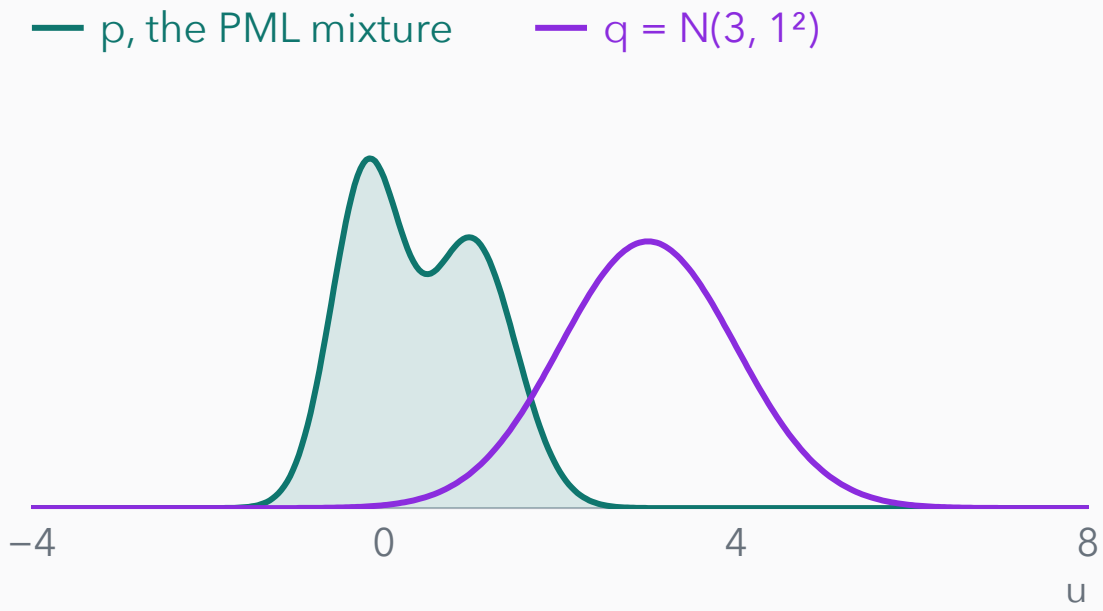
PyTorch has no formula for this KL. The definition still works.




```
kl_divergence(q, mixture)
```

Swap the target for the PML two-bump mixture and ask PyTorch for the KL to a Gaussian q.

PyTorch has no formula for this KL. The definition still works.



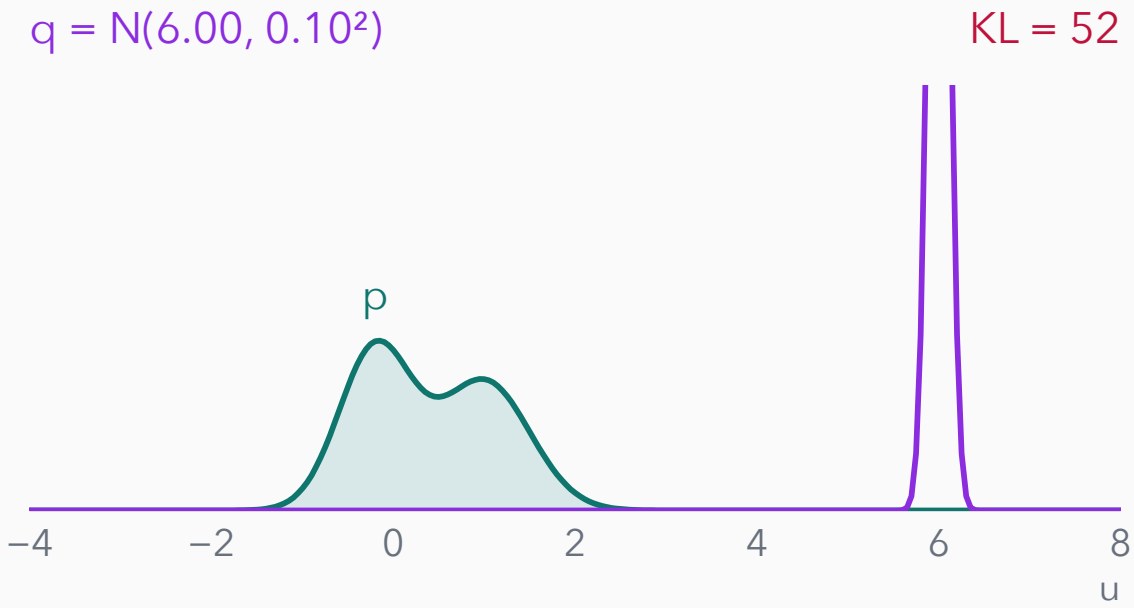

```
kl_divergence(q, mixture)
```

```
NotImplementedError:
No KL(p || q) is implemented
for p type Normal and
q type MixtureSameFamily
```

No formula. But KL is an average under q, and averages can be estimated by sampling.

Now optimise it: the code runs, and q does not move

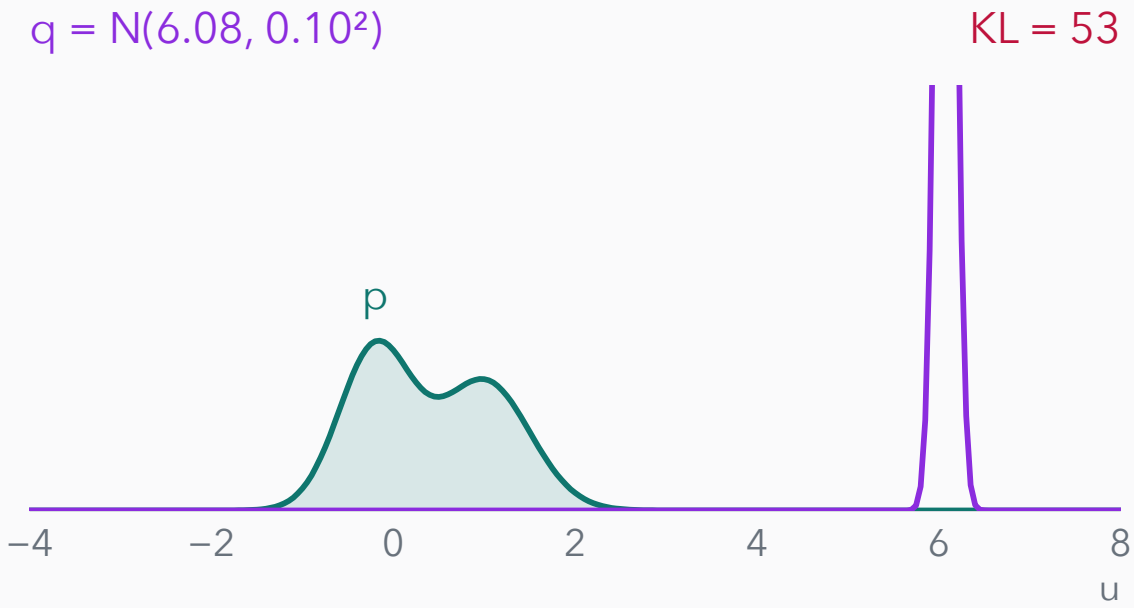
```
u = q.sample((5000,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
loss = (log_q - log_p).mean()
opt.zero_grad()
loss.backward(); opt.step()
```



Use the estimate as the loss and run the same loop on the mixture. Predict: does q move?

Now optimise it: the code runs, and q does not move

```
u = q.sample((5000,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
loss = (log_q - log_p).mean()
opt.zero_grad()
loss.backward(); opt.step()
```



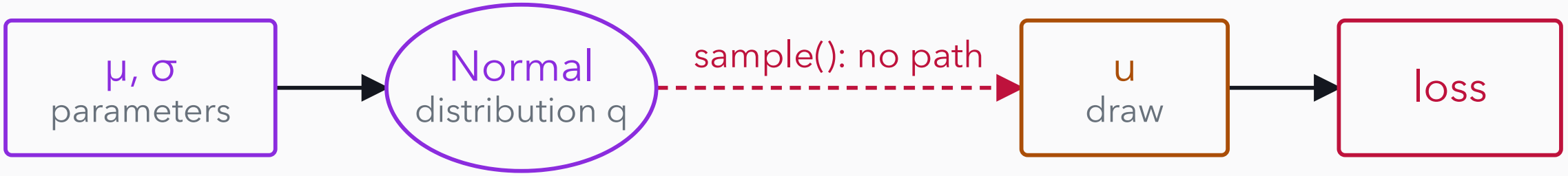
No error and no warning, and q stays at N(6, 0.1²). After 400 steps the KL is still about 53.

The gradient that would move q never reaches it



> sample() hands back plain numbers. Autograd cannot see how u would shift if μ or σ changed.

The gradient that would move q never reaches it

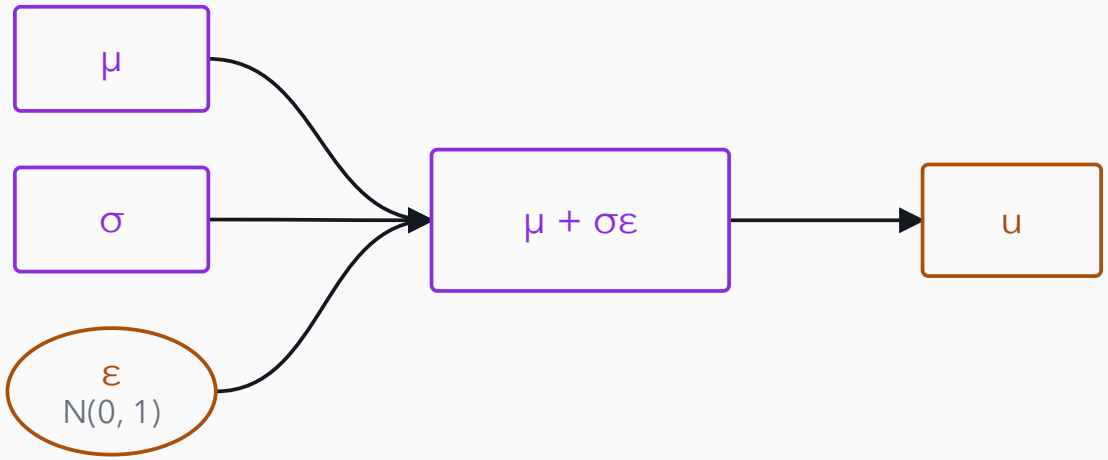


	$\partial \mathrm{KL}/\partial \mu$	$\partial \mathrm{KL}/\partial \sigma$
through sample()	0.0013	−0.0003
true gradient	−0.063	−0.375

> What survives averages to zero: 0.0013 and −0.0003, where the true gradient is −0.063 and −0.375.

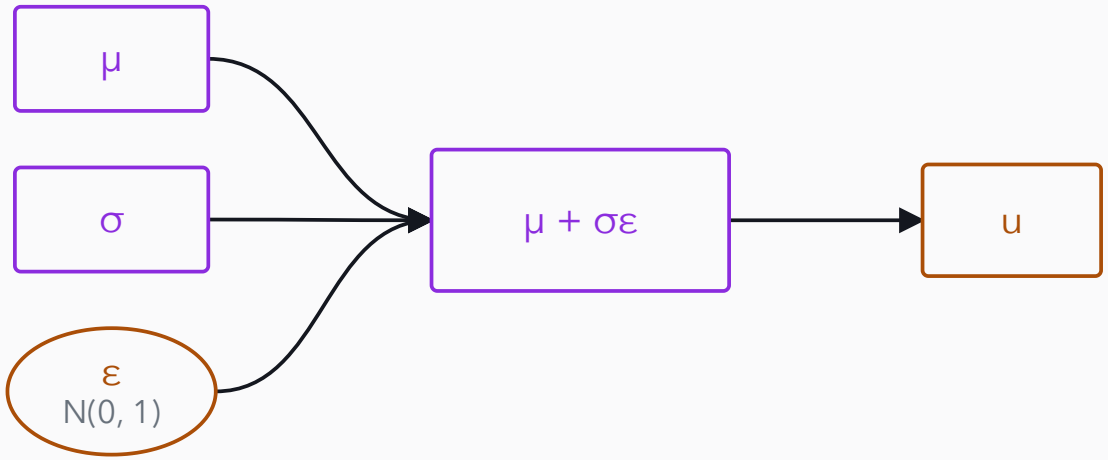
Move the randomness outside: u = μ + σε

$$\epsilon \sim \mathcal{N}(0, 1), \qquad u = \mu + \sigma\epsilon$$



> Draw the noise ε from a fixed N(0, 1), then shift and scale it. u has the same distribution, but is now a function of μ and σ.

Move the randomness outside: u = μ + σε

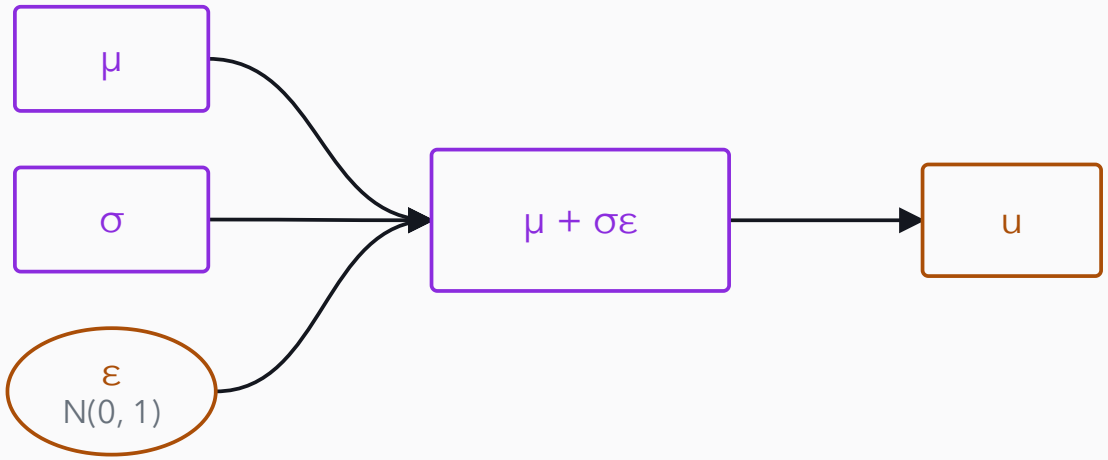


$$\epsilon \sim \mathcal{N}(0,1), \qquad u = \mu + \sigma\epsilon$$

$$\epsilon = -0.75, \quad \mu = 0, \quad \sigma = 2$$

> Pause: with ε = −0.75, μ = 0 and σ = 2, find u, ∂u/∂μ and ∂u/∂σ.

Move the randomness outside: u = μ + σε



$$\epsilon \sim \mathcal{N}(0,1), \qquad u = \mu + \sigma\epsilon$$

$$\epsilon = -0.75, \quad \mu = 0, \quad \sigma = 2$$

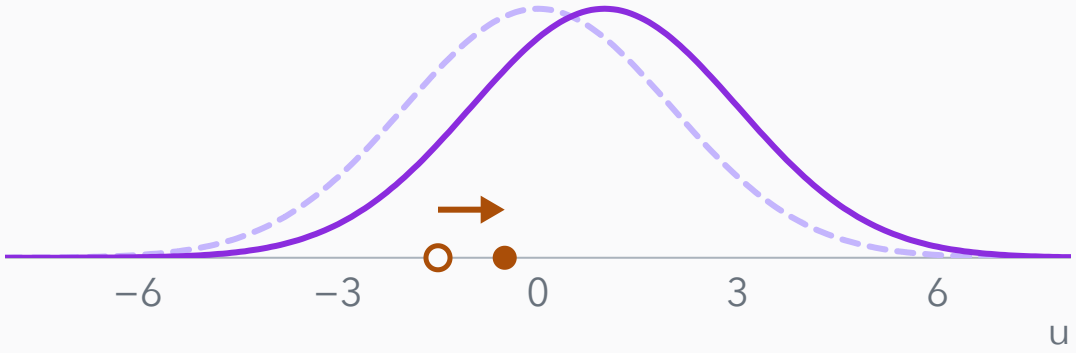
$$u = -1.5$$

$$\frac{\partial u}{\partial \mu} = 1, \qquad \frac{\partial u}{\partial \sigma} = \epsilon = -0.75$$

u = −1.5, ∂u/∂μ = 1 and ∂u/∂σ = ε = −0.75. The gradient now has a path. In PyTorch this is rsample().

What a fixed ε does when μ or σ changes

move μ from 0 to 1 u: −1.50 → −0.50



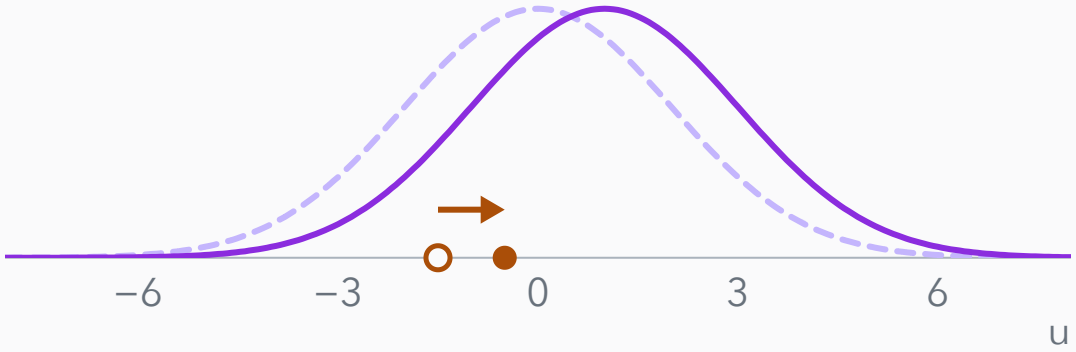
widen σ from 2 to 3 u: −1.50 → −2.25



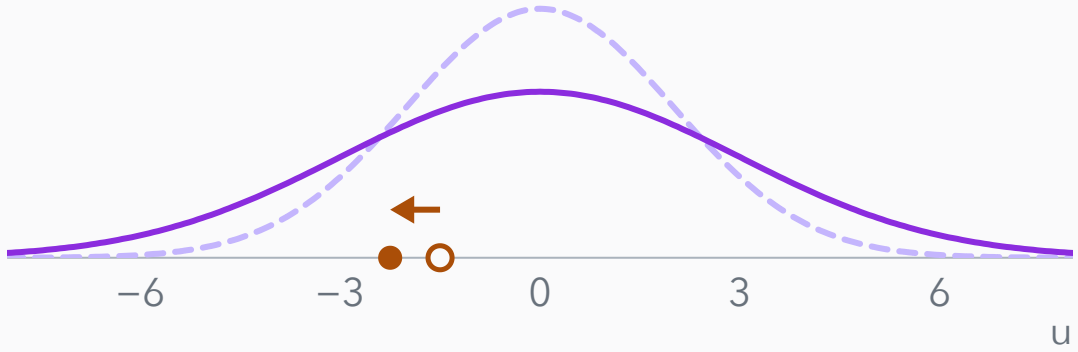
Keep the same noise draw, ε = −0.75. Shift μ by 1: the draw moves right by exactly 1.

What a fixed ε does when μ or σ changes

move μ from 0 to 1 u: −1.50 → −0.50



widen σ from 2 to 3 u: −1.50 → −2.25

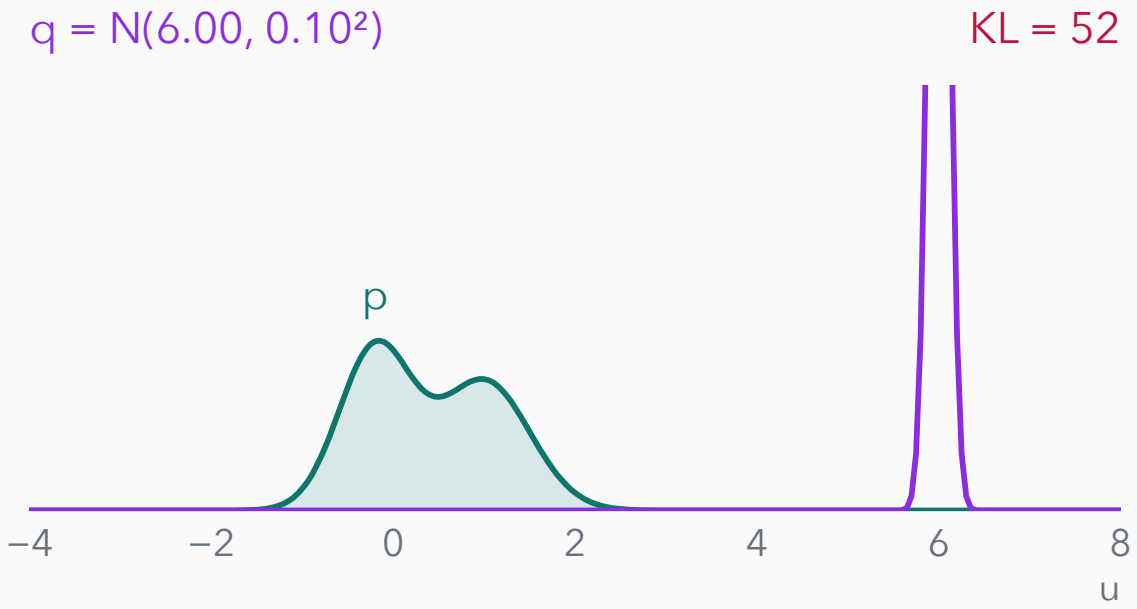


$$\frac{\partial u}{\partial \mu} = 1, \qquad \frac{\partial u}{\partial \sigma} = \epsilon = -0.75$$

> Widen σ by 1: the draw moves by ε = −0.75, further out on its side. These are the derivatives the gradient needs.

Change one word: rsample() fits the mixture

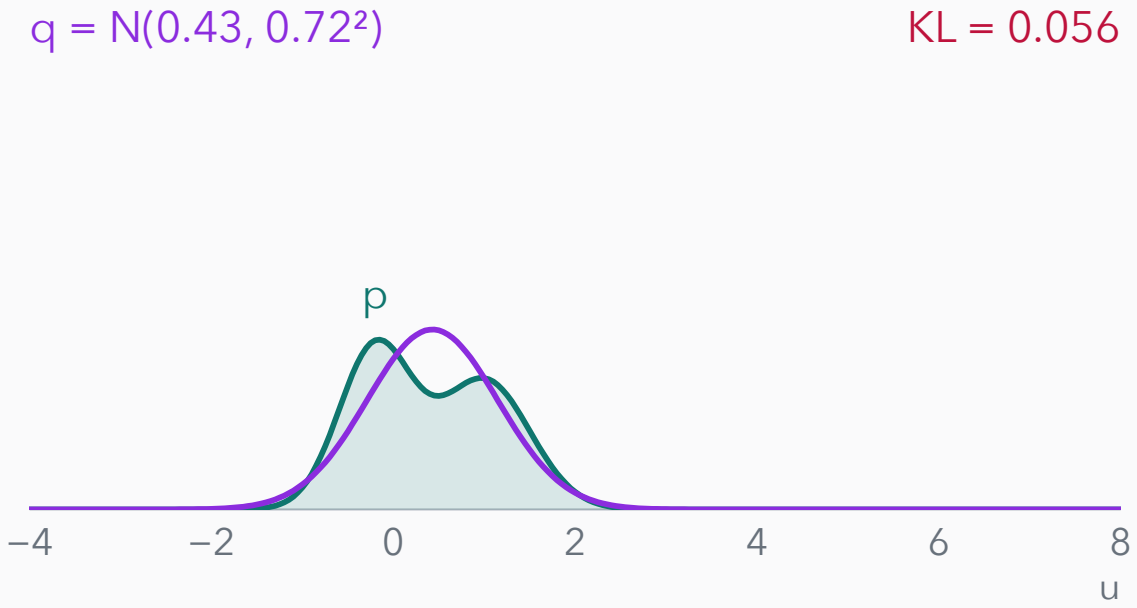
```
# one word changed
u = q.rsample((5000,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
loss = (log_q - log_p).mean()
opt.zero_grad()
loss.backward(); opt.step()
```



Same loop, same start, with q.rsample. Press play.

Change one word: rsample() fits the mixture

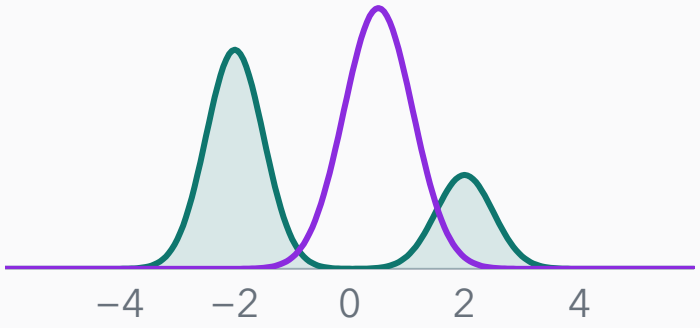
```
# one word changed
u = q.rsample((5000,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
loss = (log_q - log_p).mean()
opt.zero_grad()
loss.backward(); opt.step()
```



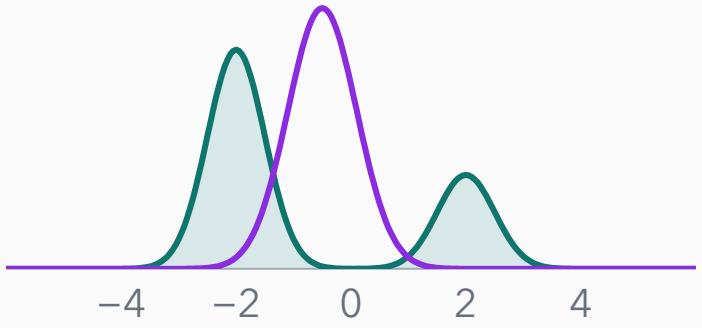
q settles at N(0.43, 0.72²), KL = 0.056: the best single Gaussian for this mixture. No formula was needed.

The price: reverse KL picks one bump

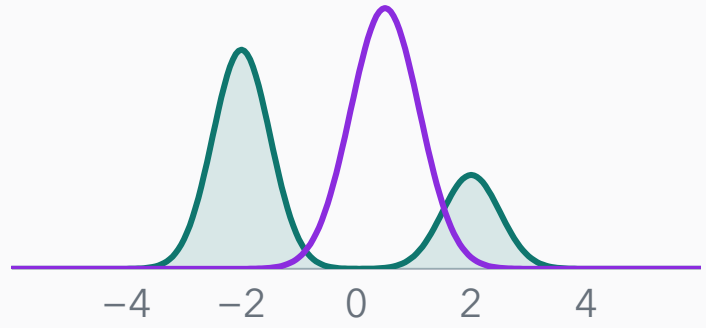
KL(pllq) 6.43



KL(qllp), start left 3.92



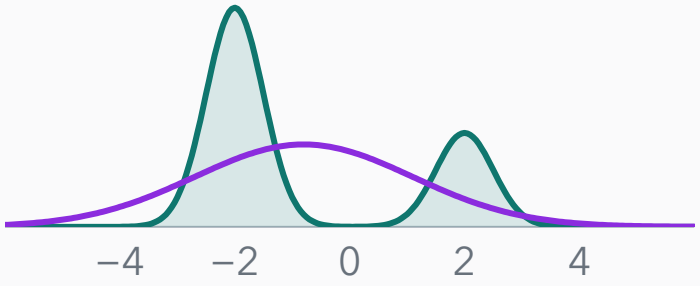
KL(qllp), start right 4.42



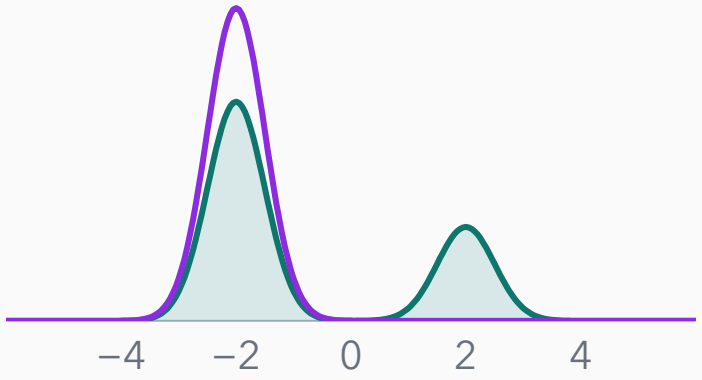
Fit one Gaussian to two narrow bumps, three times. Predict where each q settles.

The price: reverse KL picks one bump

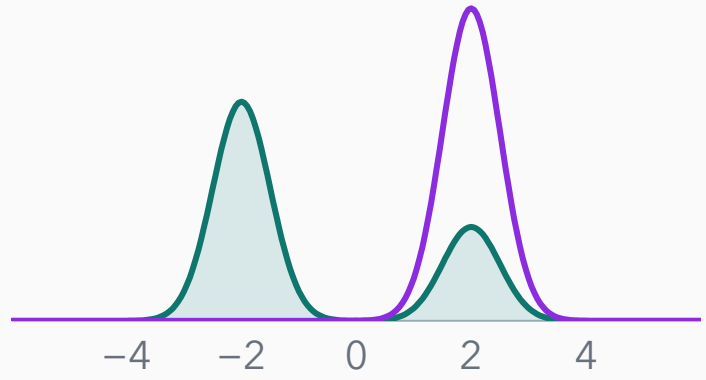
KL(p||q) 0.72



KL(q||p), start left 0.36



KL(q||p), start right 1.20



KL(p||q) spreads over both bumps. KL(q||p) locks onto whichever bump is nearer its start, and reports it confidently.

Back to Bayes: minimise KL to the posterior directly?

$$\mathrm{KL}\big(q_\psi(\theta) \,\|\, p(\theta \mid \mathcal{D})\big) = \mathbb{E}_{\theta\sim q}\big[\log q_\psi(\theta) - \log p(\theta \mid \mathcal{D})\big]$$

> The plan from Part 1: make q close to the posterior. Use KL(q ‖ posterior): it averages under q, which we can sample.

Back to Bayes: minimise KL to the posterior directly?

$$\mathrm{KL}\big(q_\psi(\theta) \,\|\, p(\theta \mid \mathcal{D})\big) = \mathbb{E}_{\theta\sim q}\big[\log q_\psi(\theta) - \log p(\theta \mid \mathcal{D})\big]$$

the loss needs	available?
draws θ ~ q	yes: rsample, as in Part 3
$\log q_\psi(\theta)$	yes
$\log p(\theta \mid \mathcal{D}) = \log p(\mathcal{D} \mid \theta) + \log p(\theta) - \log p(\mathcal{D})$	no: needs p($\mathcal{D}$)

> Everything is available except log p(θ | $\mathcal{D}$), which needs the evidence p($\mathcal{D}$): the integral we cannot do.

Substitute Bayes' rule and see what survives

$$\mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big) = \mathbb{E}_q\big[\log q(\theta) - \log p(\theta \mid \mathcal{D})\big]$$

Pause: substitute log p(θ | $\mathcal{D}$) = log p($\mathcal{D}$ | θ) + log p(θ) − log p($\mathcal{D}$). What happens to log p($\mathcal{D}$)?

Substitute Bayes' rule and see what survives

$$\mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big) = \mathbb{E}_q\big[\log q(\theta) - \log p(\theta \mid \mathcal{D})\big]$$

$$= \mathbb{E}_q\big[\log q(\theta) - \log p(\mathcal{D} \mid \theta) - \log p(\theta)\big] + \log p(\mathcal{D})$$

> It does not depend on θ, so it leaves the average as a constant.

Substitute Bayes' rule and see what survives

$$\mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big) = \mathbb{E}_q\big[\log q(\theta) - \log p(\theta \mid \mathcal{D})\big]$$

$$= \mathbb{E}_q\big[\log q(\theta) - \log p(\mathcal{D} \mid \theta) - \log p(\theta)\big] + \log p(\mathcal{D})$$

$$\log p(\mathcal{D}) = \underbrace{\mathbb{E}_q\big[\log p(\mathcal{D} \mid \theta) + \log p(\theta) - \log q(\theta)\big]}_{\mathrm{ELBO}(q)} + \mathrm{KL}\big(q \,\|\, p(\theta \mid \mathcal{D})\big)$$

Rearrange: log p($\mathcal{D}$) = ELBO + KL. The ELBO needs only the likelihood, the prior and q.

The ELBO for our network: fit the data, stay near the prior

$$\mathrm{ELBO}(q) = \mathbb{E}_q\big[\log p(\mathcal{D} \mid \theta)\big] + \mathbb{E}_q\big[\log p(\theta) - \log q(\theta)\big]$$

$$= \underbrace{\mathbb{E}_q\big[\log p(\mathcal{D} \mid \theta)\big]}_{\text{fit the data}} - \underbrace{\mathrm{KL}\big(q(\theta) \,\|\, p(\theta)\big)}_{\text{stay near the prior}}$$

> Group the terms: the ELBO rewards fitting the data and charges for moving q away from the prior.

The ELBO for our network: fit the data, stay near the prior

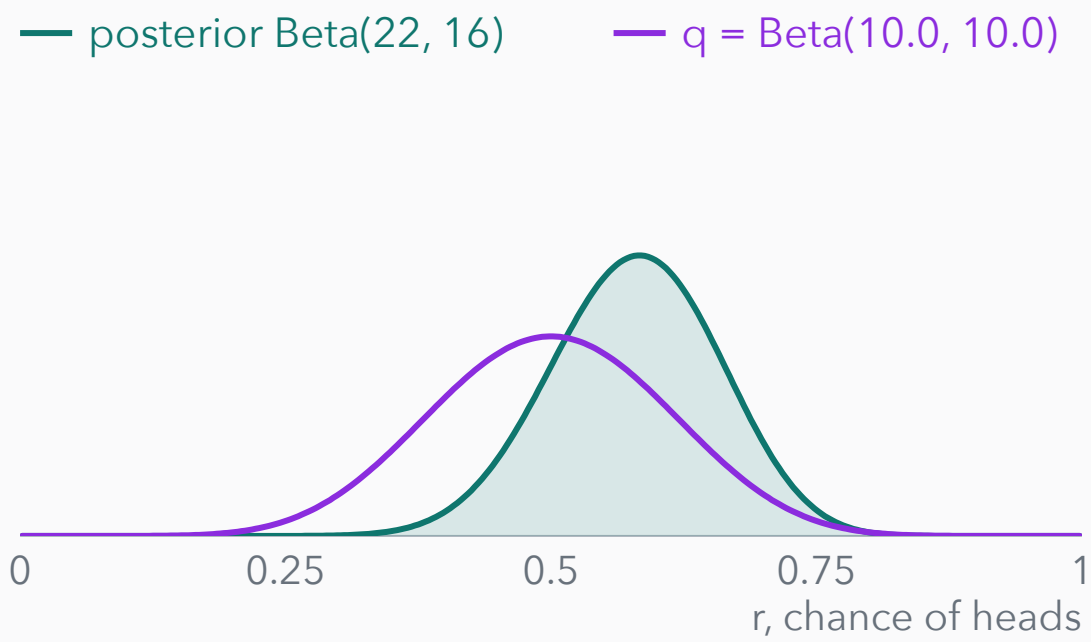
$$\mathrm{ELBO}(q) = \mathbb{E}_q\big[\log p(\mathcal{D} \mid \theta)\big] + \mathbb{E}_q\big[\log p(\theta) - \log q(\theta)\big]$$

$$= \underbrace{\mathbb{E}_q\big[\log p(\mathcal{D} \mid \theta)\big]}_{\text{fit the data}} - \underbrace{\mathrm{KL}\big(q(\theta) \,\|\, p(\theta)\big)}_{\text{stay near the prior}}$$

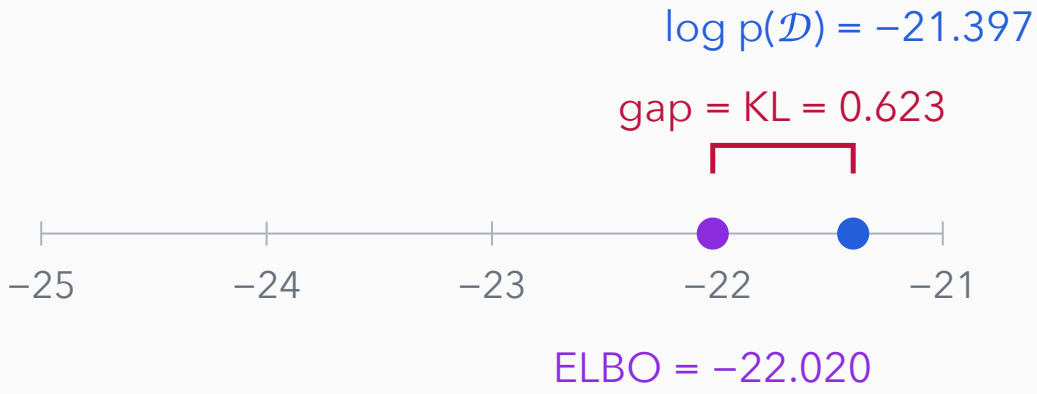
The second term is the diagonal-Gaussian KL from Part 2: a formula, seven terms. The first is a Monte Carlo average over networks sampled with rsample.

> The KL to the prior has a formula; the data term is estimated by sampling θ from q, with the reparameterisation trick.

Check the identity where p($\mathcal{D}$) is known: a coin



One fixed total



−22.020 + 0.623 = −21.397

A coin with a Beta(5, 3) prior and 17 heads in 30 tosses has an exact posterior and an exact p($\mathcal{D}$). Drag q: the ELBO and the KL trade off, and their sum never changes.

The ELBO for our network, in code


```
mu  = Parameter(0.5 * randn(7))          # 7 means
rho = Parameter(full((7,), log(0.1)))    # 7 log-widths
q = Normal(mu, rho.exp())                # one bell per parameter
theta = q.rsample((16,))                 # 16 sampled networks
log_lik = log_likelihood(theta).mean()   # forward pass
kl = kl_divergence(q, prior).sum()       # closed form
loss = -(log_lik - kl)                   # minus the ELBO
```

Fourteen parameters, sixteen sampled networks per step, and the closed-form KL to the prior.

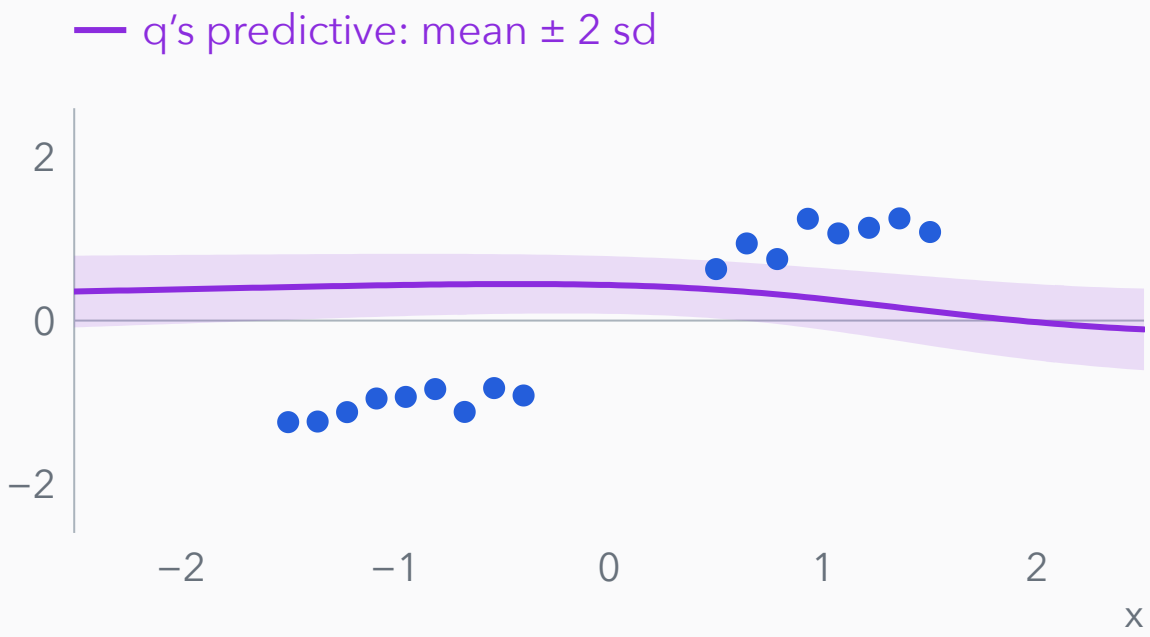
The ELBO for our network, in code

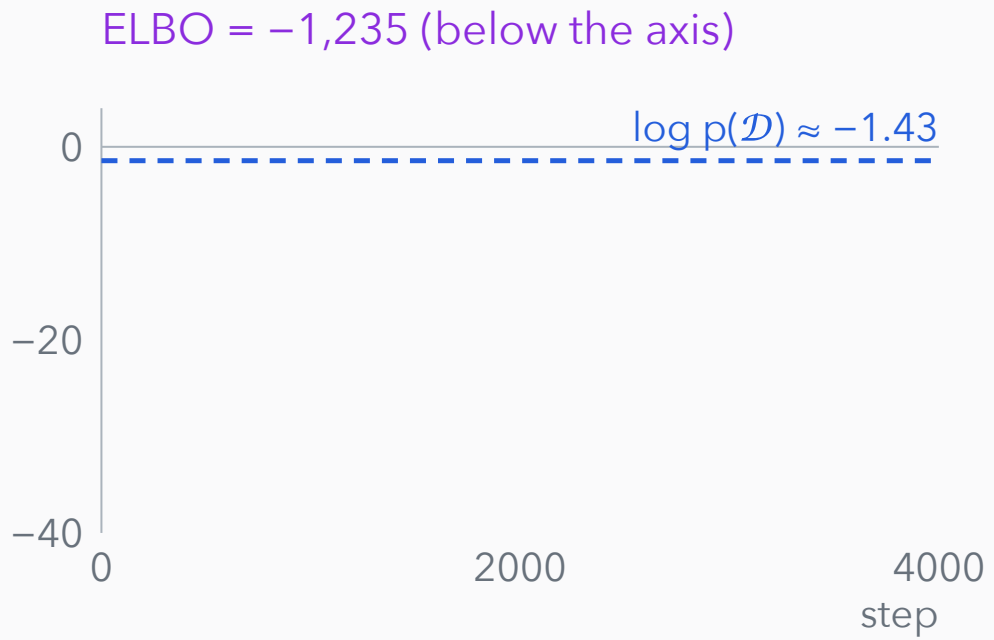

```
mu  = Parameter(0.5 * randn(7))          # 7 means
rho = Parameter(full((7,), log(0.1)))    # 7 log-widths
q = Normal(mu, rho.exp())                # one bell per parameter
theta = q.rsample((16,))                 # 16 sampled networks
log_lik = log_likelihood(theta).mean()   # forward pass
kl = kl_divergence(q, prior).sum()       # closed form
loss = -(log_lik - kl)                   # minus the ELBO
```

Adam, learning rate 0.02, 4,000 steps; best of five random starts. No p($\mathcal{D}$) anywhere.

Every line is computable: rsample, a forward pass and a formula. Never p($\mathcal{D}$).

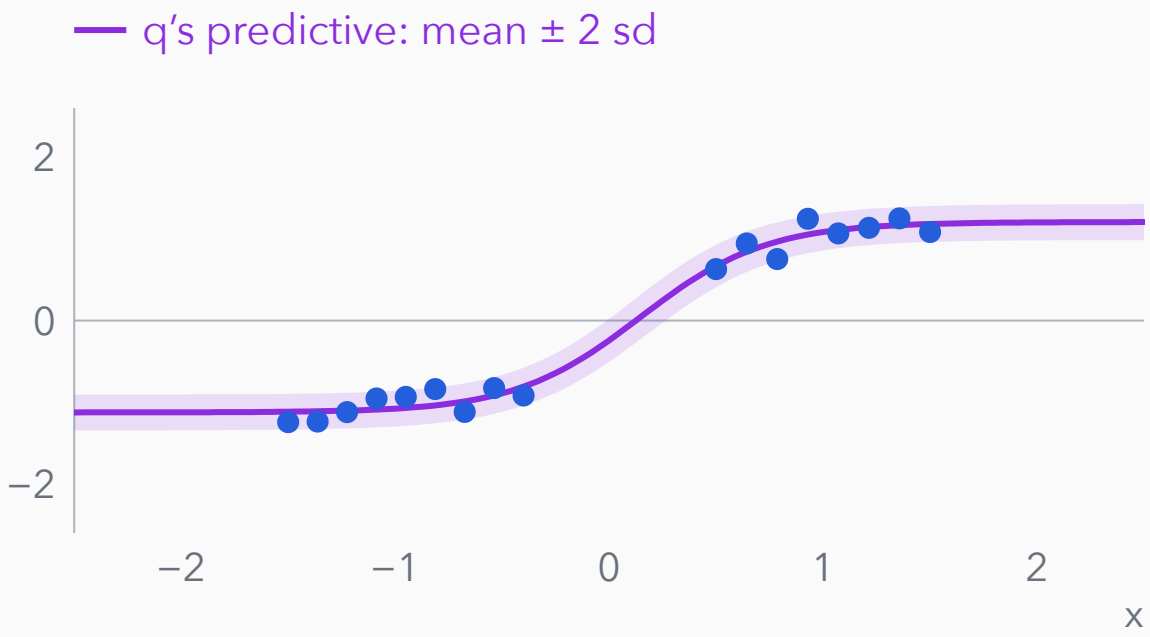
Press play: q learns the network

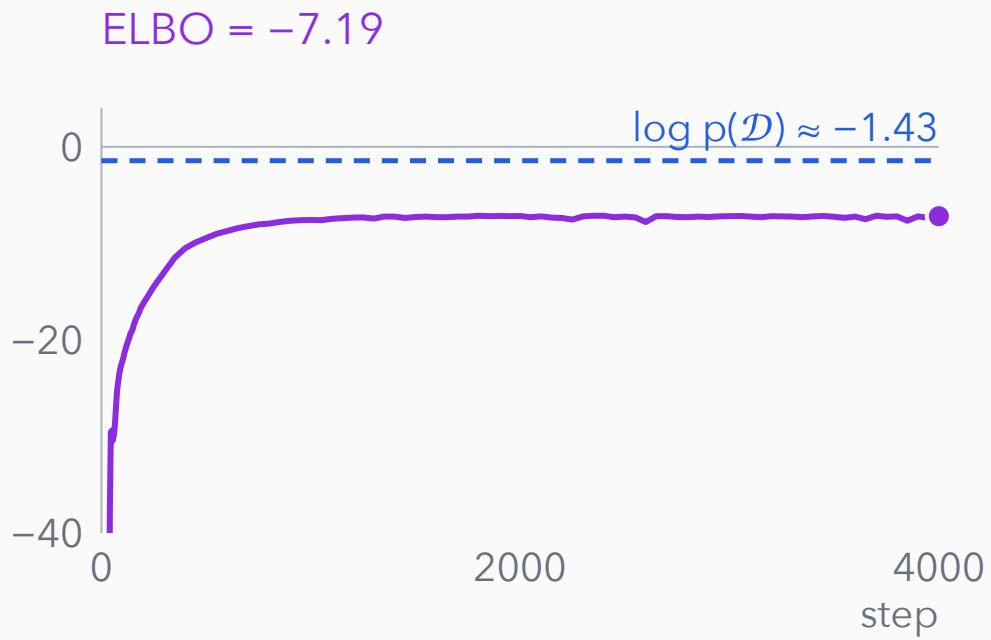




Start from random means and narrow widths. Watch q's predictive band and the ELBO.

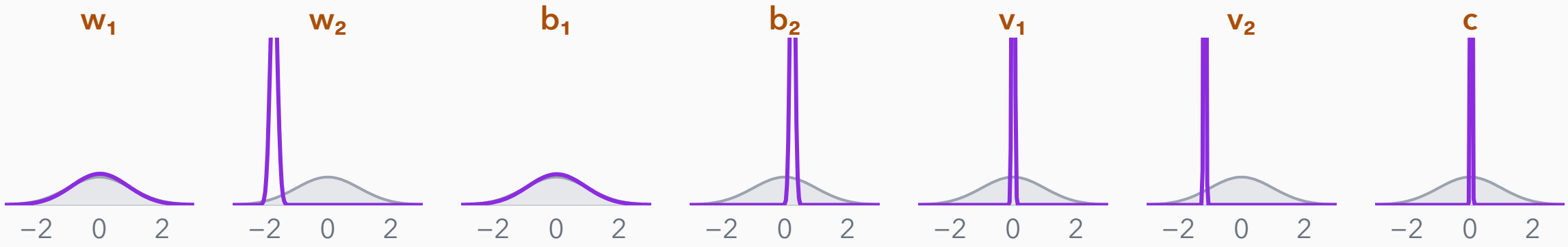
Press play: q learns the network





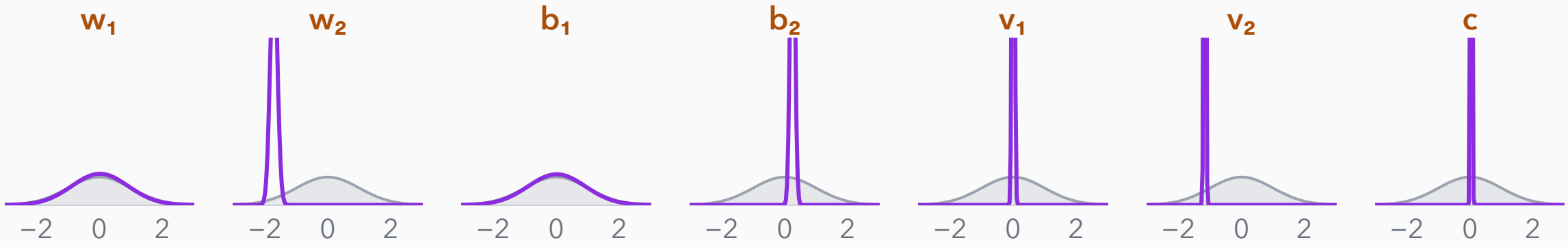
> The ELBO climbs from −1,235 to about −7.2 and the band settles onto the data. It stays below log p(𝒟) ≈ −1.43.

What q learned: one hidden unit switched off



The fitted bell for each parameter, against the N(0, 1) prior in grey.

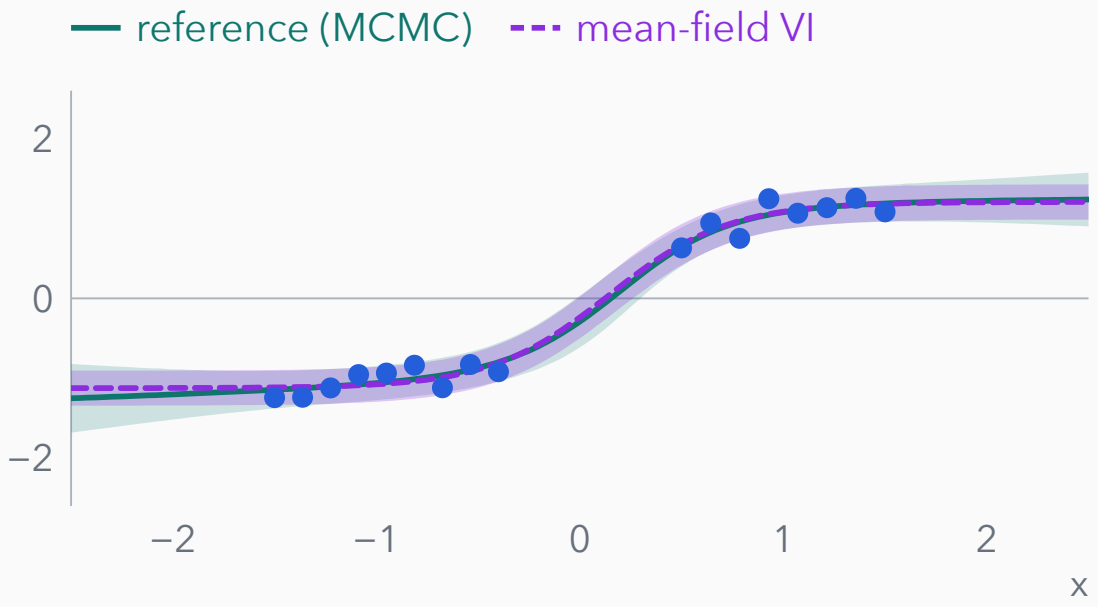
What q learned: one hidden unit switched off



$v_1 \approx 0.01$ with width 0.035: the first hidden unit is switched off, and its w_1 and b_1 fall back to the prior.

> VI switched the first hidden unit off. Mean-field VI often prunes units like this: every parameter kept away from the prior costs KL.

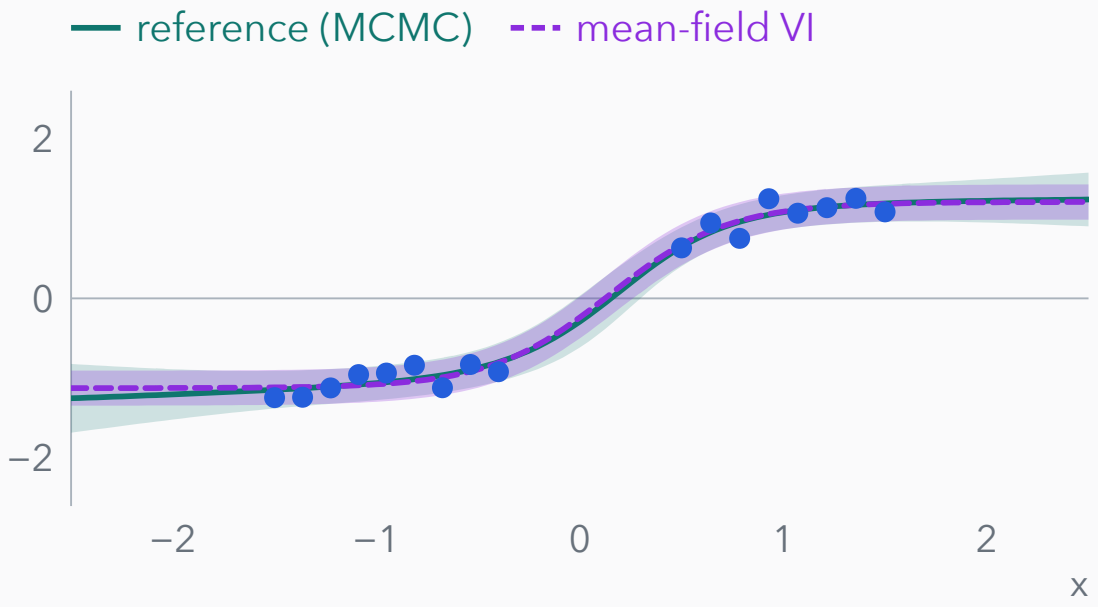
VI against the reference: the right curve, too-narrow bands



predictive sd	VI	reference
at x = −2.5	0.11	0.22
at x = 2.5	0.11	0.17

> Mean-field VI gets the mean curve right.

VI against the reference: the right curve, too-narrow bands



predictive sd	VI	reference
at x = −2.5	0.11	0.22
at x = 2.5	0.11	0.17

ELBO −7.18 against log p($\mathcal{D}$) ≈ −1.43: q is about 5.8 nats from the posterior.

> Its bands are too narrow beyond the data, as the diagonal q in 2-D predicted. The ELBO gap says how far q is from the posterior.

Six obstacles, six tools

obstacle	tool	in our network
$p(\mathcal{D})$ is an impossible integral	approximate with q	one Gaussian per parameter
“close” needs a number	$KL(q \Vert p)$	KL to the prior
a grid cannot search 14 numbers	gradient descent	Adam on μ and ρ
no formula for the average	Monte Carlo	16 sampled networks
no gradient through a draw	$\theta = \mu + \sigma\varepsilon$, rsample()	reparameterisation
KL to the posterior needs $p(\mathcal{D})$	maximise the ELBO	the training loss

> Every obstacle today earned a tool, and our network used them all. In Lecture 3 a VAE does the same with one q per image.

Generative models create this inference problem again

A hidden description can generate an observation.

> In an image model, a hidden code z can generate an observed image x.

Generative models create this inference problem again

A hidden description can generate an observation.

Next: learn representations for images, then ask whether they can generate new ones.

> Lecture 2 builds autoencoders; Lecture 3 returns to VI with an encoder that outputs q for each image.

A sensor has an unknown constant bias

$$u \sim \mathcal{N}(0, 1), \qquad y_i \mid u \sim \mathcal{N}(u, 0.5^2)$$

A sensor reads a hidden bias u, with noise. Name the unknown, the data and a q.

A sensor has an unknown constant bias

$$u \sim \mathcal{N}(0, 1), \qquad y_i \mid u \sim \mathcal{N}(u, 0.5^2)$$

$$p(\mathcal{D}, u) = p(u) \prod_i p(y_i \mid u)$$

> The joint is the prior times the likelihood of each reading: available.

A sensor has an unknown constant bias

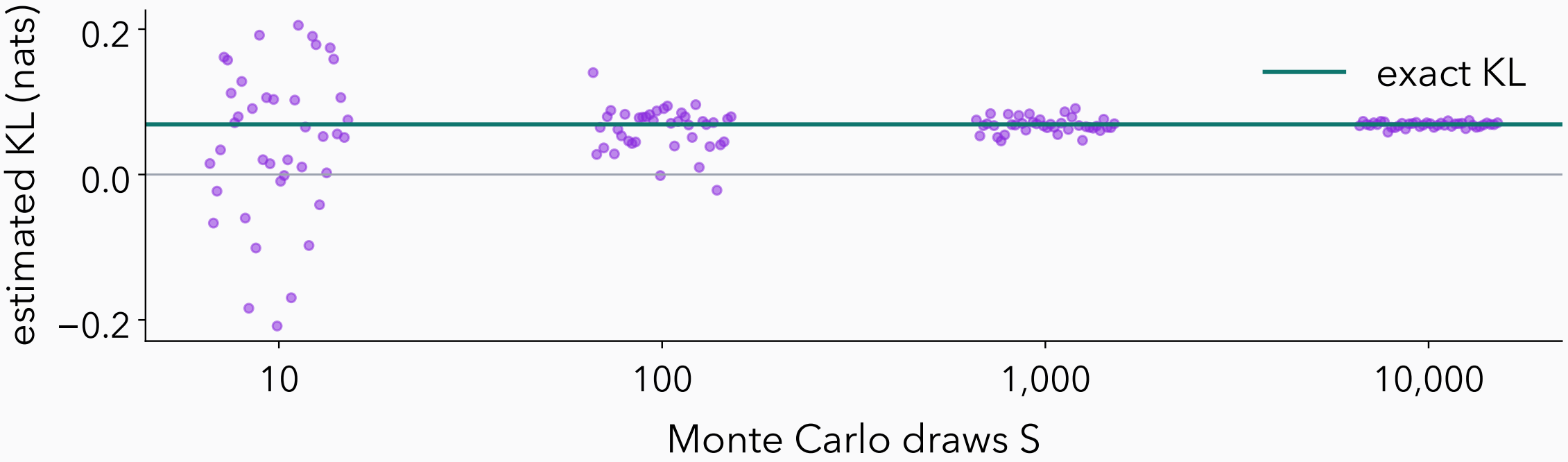
$$u \sim \mathcal{N}(0, 1), \qquad y_i \mid u \sim \mathcal{N}(u, 0.5^2)$$

$$p(\mathcal{D}, u) = p(u) \prod_i p(y_i \mid u)$$

$$\max_{\psi} \mathbb{E}_{q_\psi(u)}[\log p(\mathcal{D}, u) - \log q_\psi(u)]$$

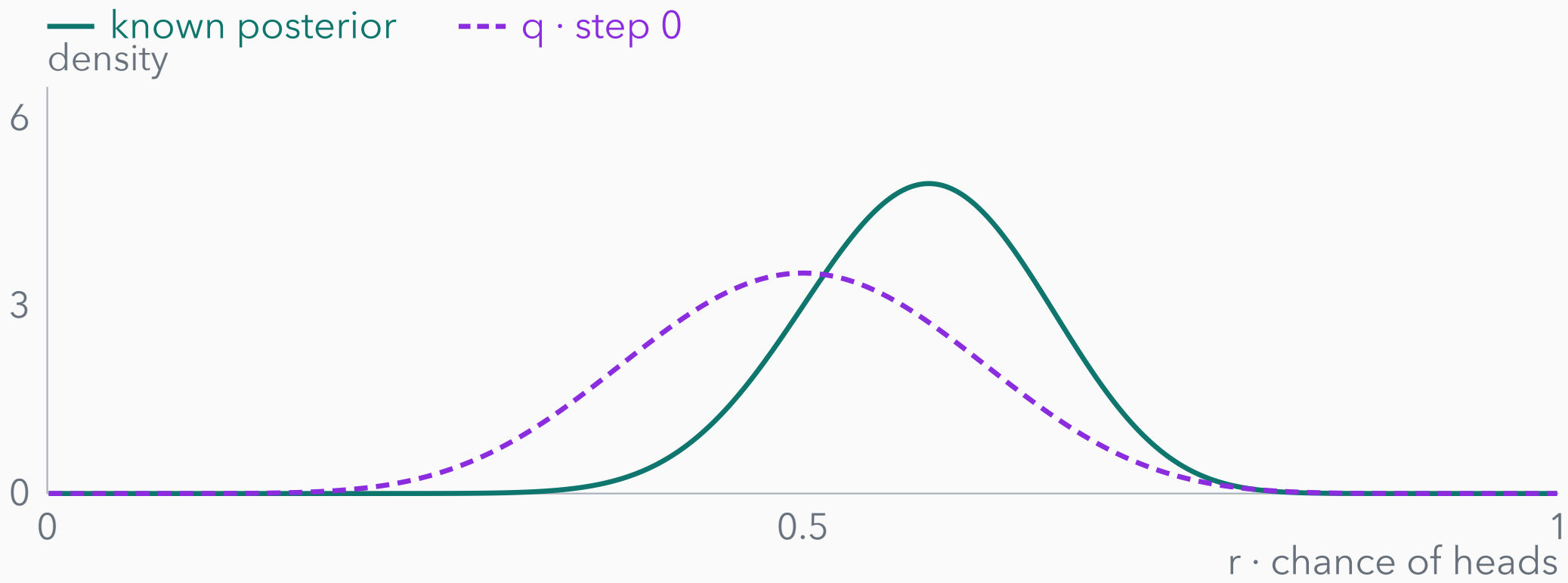
Maximise the ELBO over q's parameters. The same tools apply.

More samples, tighter estimates



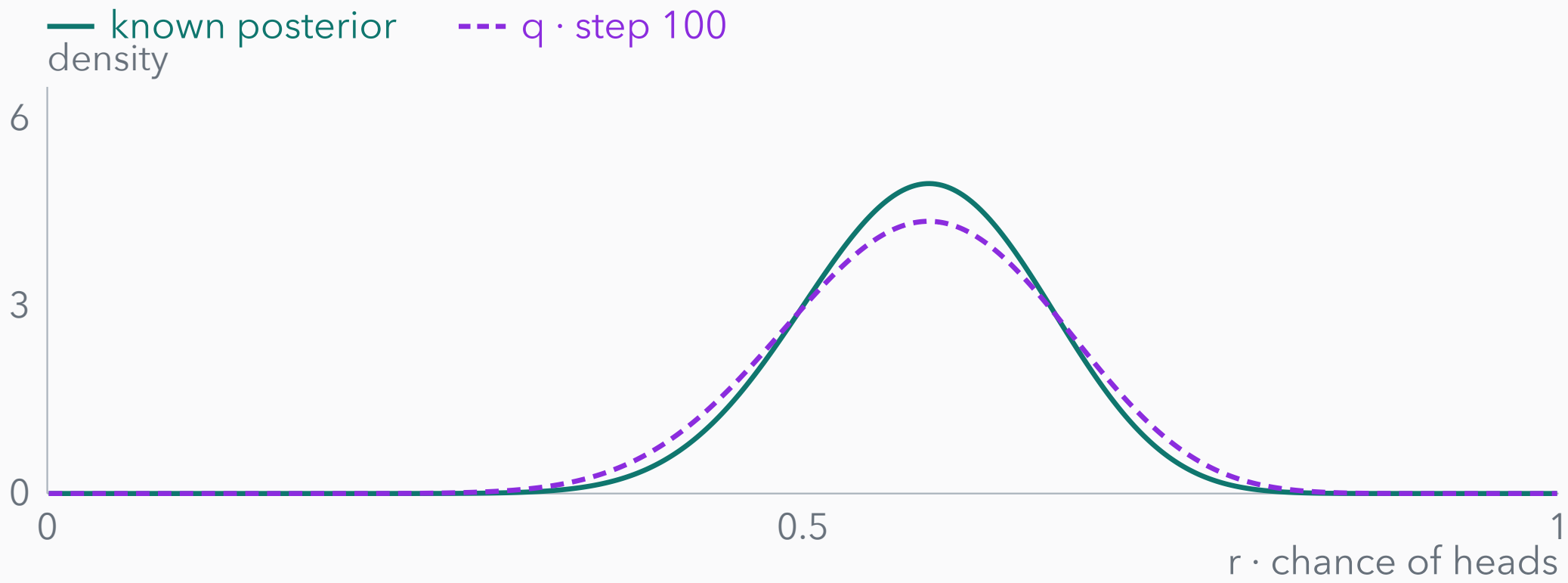
Forty repeats at each sample size. The spread shrinks like $1/\sqrt{S}$; the exact value never moves.

A check with an answer key: the Beta coin



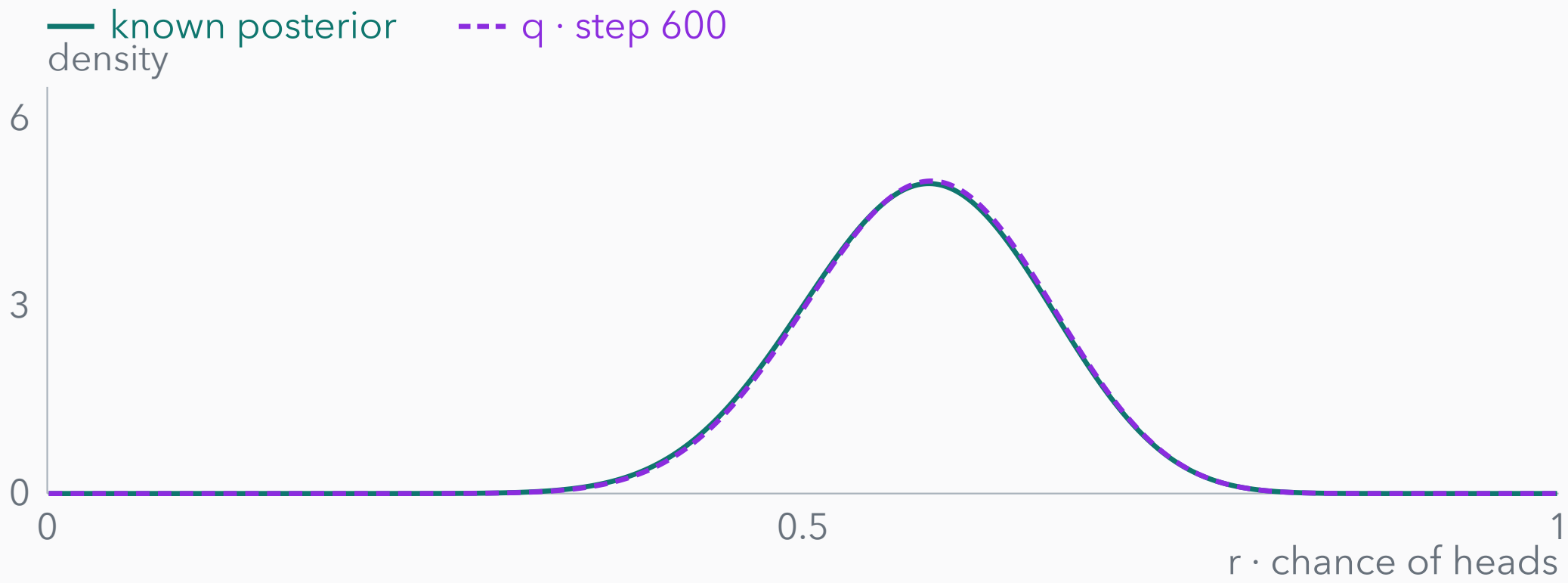
With the Beta(5, 3) prior we know the posterior: Beta(22, 16). Fit a Beta q from Beta(10, 10) using only the joint.

A check with an answer key: the Beta coin



By step 100, q has moved most of the way.

A check with an answer key: the Beta coin



After 600 steps: Beta(22.4, 16.2), with KL ≈ 0.0002 to the true posterior.

One literal update of the mean

$$\sigma = 2 \text{ fixed}: \qquad L(\mu) = \text{const} + \frac{(\mu - 1)^2}{32}$$

> Which term of the Gaussian KL depends on μ?

One literal update of the mean

$$\sigma = 2 \text{ fixed}: \qquad L(\mu) = \text{const} + \frac{(\mu - 1)^2}{32}$$

$$\frac{\partial L}{\partial \mu} = \frac{\mu - 1}{16} = -0.0625 \ \text{ at } \mu = 0$$

Differentiate it at μ = 0.

One literal update of the mean

$$\sigma = 2 \text{ fixed}: \qquad L(\mu) = \text{const} + \frac{(\mu - 1)^2}{32}$$

$$\frac{\partial L}{\partial \mu} = \frac{\mu - 1}{16} = -0.0625 \ \text{ at } \mu = 0$$

$$\mu^{+} = 0 - 4(-0.0625) = 0.25: \quad \text{KL } 0.349 \to 0.336$$

Take one step with η = 4. What is the best KL with σ stuck at 2?

KL ≥ 0 by Jensen’s inequality

$$\mathrm{KL}(q\|p) = -\mathbb{E}_q\left[\log \frac{p(u)}{q(u)}\right]$$

Which function makes Jensen’s inequality useful?

KL ≥ 0 by Jensen's inequality

$$\mathrm{KL}(q\|p) = -\mathbb{E}_q\left[\log \frac{p(u)}{q(u)}\right]$$

$$\geq -\log \mathbb{E}_q\left[\frac{p(u)}{q(u)}\right]$$

Why does the inequality point this way?

KL ≥ 0 by Jensen's inequality

$$\mathrm{KL}(q\|p) = -\mathbb{E}_q\left[\log \frac{p(u)}{q(u)}\right]$$

$$\geq -\log \mathbb{E}_q\left[\frac{p(u)}{q(u)}\right]$$

$$= -\log 1 = 0$$

What makes the final expectation one?

Five draws can give a negative KL estimate

u_i	$\log q(u_i) - \log p(u_i)$
5.623	−0.232
0.120	0.269
−5.536	−0.751
2.705	0.217
−2.254	0.030

> Some rows are negative. Could their average be negative?

Five draws can give a negative KL estimate

u_i	$\log q(u_i) - \log p(u_i)$
5.623	−0.232
0.120	0.269
−5.536	−0.751
2.705	0.217
−2.254	0.030

$$\widehat{\mathrm{KL}}_5 \approx -0.093, \qquad \mathrm{KL}(q\|p) \approx 0.069$$

> Does this contradict KL ≥ 0?

The two PML candidates, scored by Monte Carlo

candidate	MC estimate ± 1 s.e.	numerical integral
$N(3, 1^2)$	9.54 ± 0.03	9.49
$N(3, 4.5^2)$	45.02 ± 0.21	45.15

> How large is the difference relative to the sampling uncertainty?

The two PML candidates, scored by Monte Carlo

candidate	MC estimate ± 1 s.e.	numerical integral
$N(3, 1^2)$	9.54 ± 0.03	9.49
$N(3, 4.5^2)$	45.02 ± 0.21	45.15

The narrow candidate scores far better. Both sit away from the mixture's mass, so both are poor.

> What role does the numerical integral play?

A fixed-noise finite difference checks the path

```
q = Normal(mu, sigma)
u = q.rsample((S,))
loss = (q.log_prob(u) - p.log_prob(u)).mean()
```

What is different about rsample?

A fixed-noise finite difference checks the path

```
q = Normal(mu, sigma)
u = q.rsample((S,))
loss = (q.log_prob(u) - p.log_prob(u)).mean()
```

derivative	autograd	finite difference
μ	−0.062466	−0.062466
σ	−0.372223	−0.372223

What must stay fixed on both sides of the finite difference?

Pathwise gradients are one route, not the only one

$$\nabla_\psi \mathbb{E}_{q_\psi}[f_\psi(u)] = \mathbb{E}_{q_\psi}[f_\psi(u)\nabla_\psi \log q_\psi(u) + \nabla_\psi f_\psi(u)]$$

Which term was missing from the naive sample() gradient?

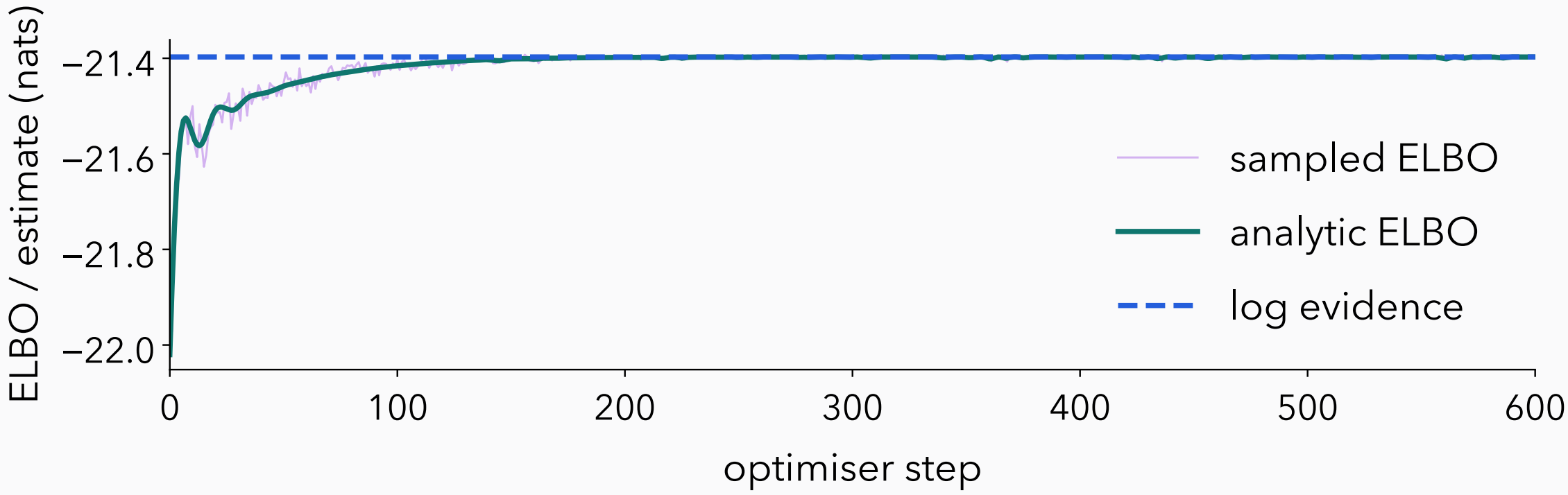
Pathwise gradients are one route, not the only one

$$\nabla_\psi \mathbb{E}_{q_\psi}[f_\psi(u)] = \mathbb{E}_{q_\psi}[f_\psi(u)\nabla_\psi \log q_\psi(u) + \nabla_\psi f_\psi(u)]$$

A score-function estimator accounts explicitly for the changing sampling distribution.

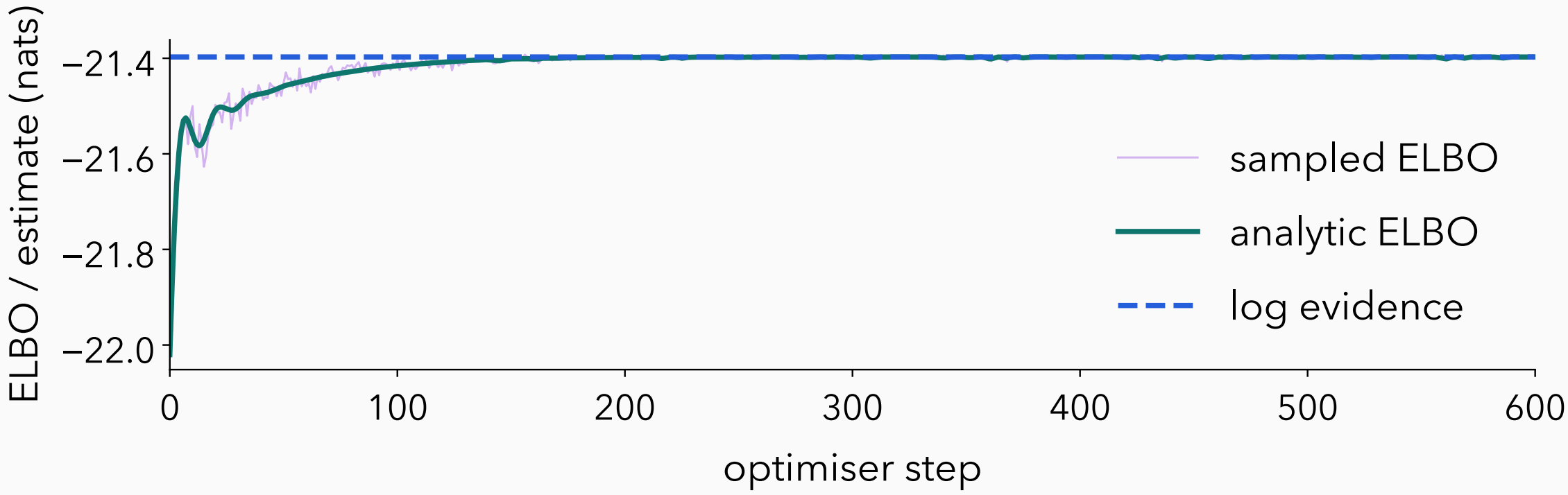
> Does this identity require u = μ + σε?

A noisy ELBO estimate can cross the evidence



Can a finite-sample ELBO estimate cross the reference line?

A noisy ELBO estimate can cross the evidence



162 of 600 sampled estimates lie above the known evidence in this run.

> Which object remains a lower bound?

A higher ELBO does not prove a zero gap

$$\mathrm{ELBO}(q_{\mathrm{new}}) > \mathrm{ELBO}(q_{\mathrm{old}})$$

What does a true increase in the exact ELBO establish?

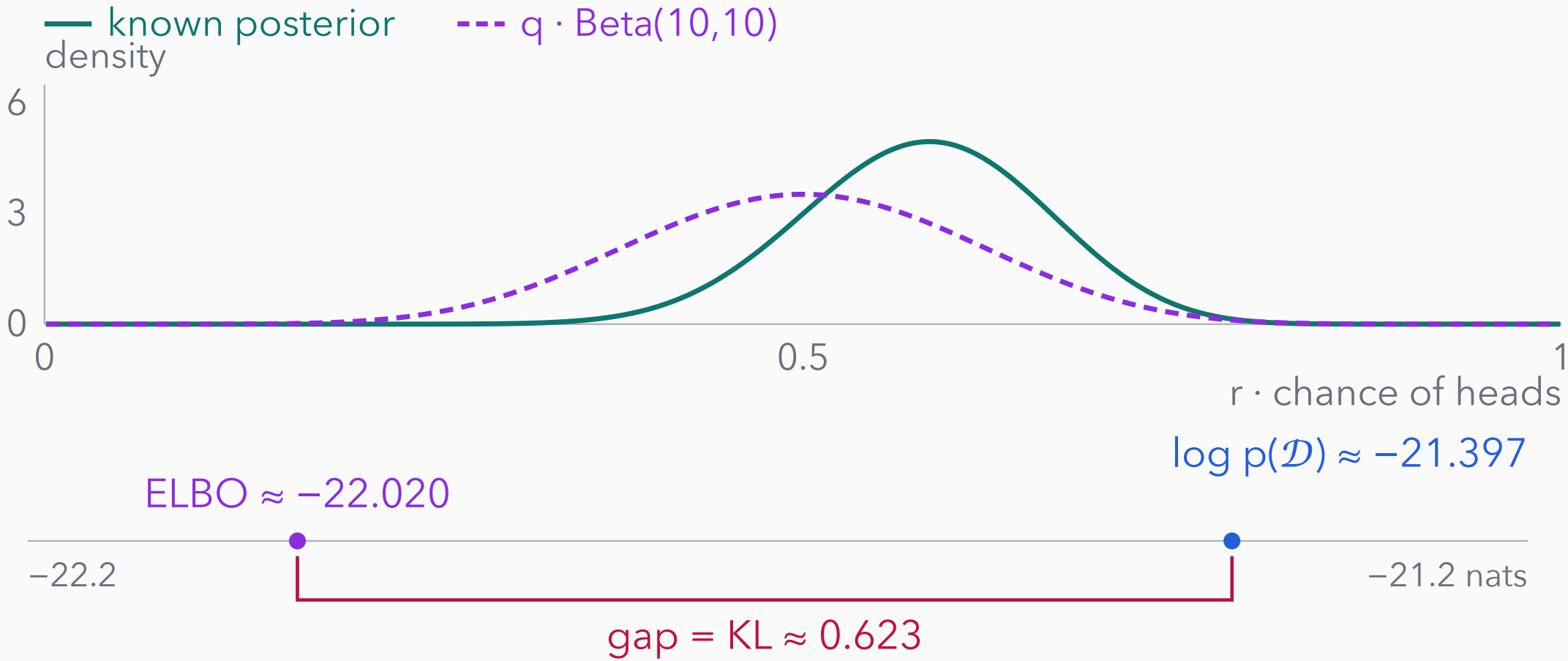
A higher ELBO does not prove a zero gap

$$\mathrm{ELBO}(q_{\mathrm{new}}) > \mathrm{ELBO}(q_{\mathrm{old}})$$

The family may not contain the posterior, and optimisation may stop short.

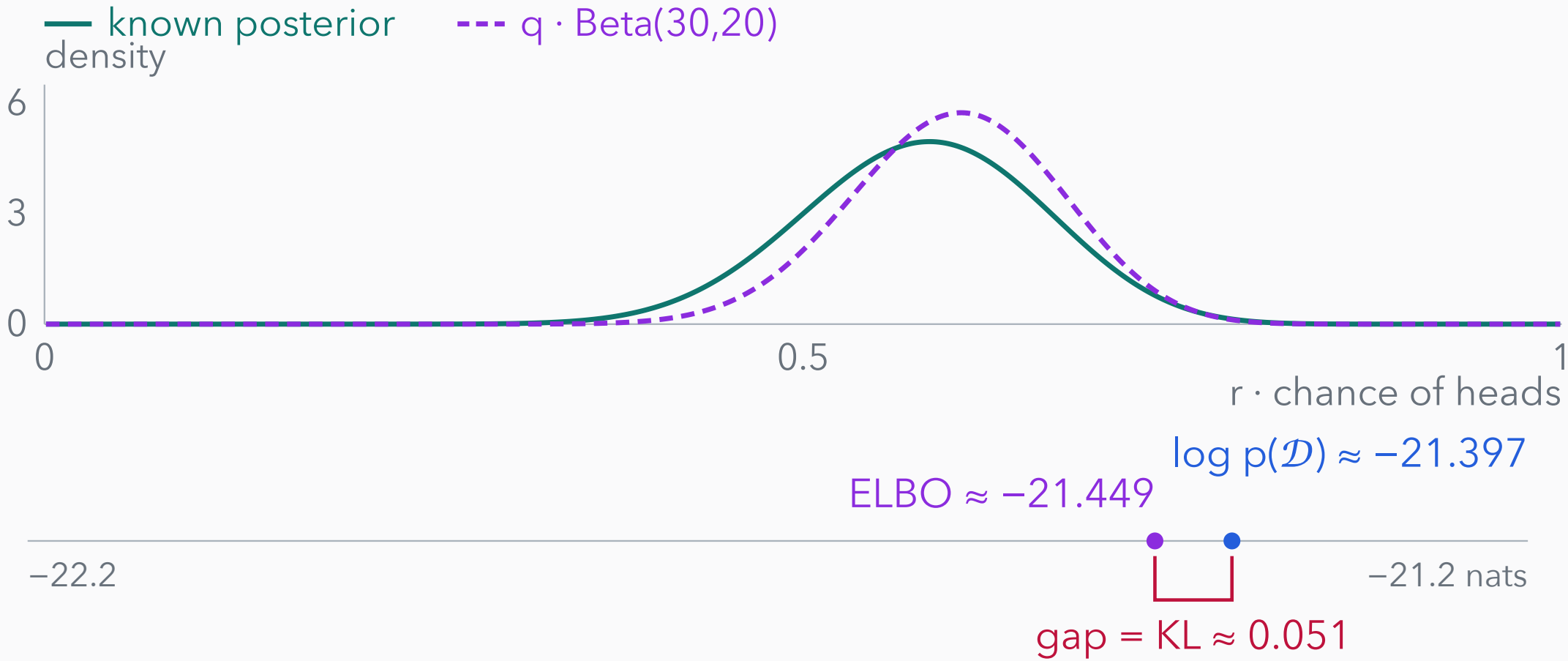
> Does a stable loss establish q = p(u | $\mathcal{D}$)?

Check the fixed total with three Beta candidates



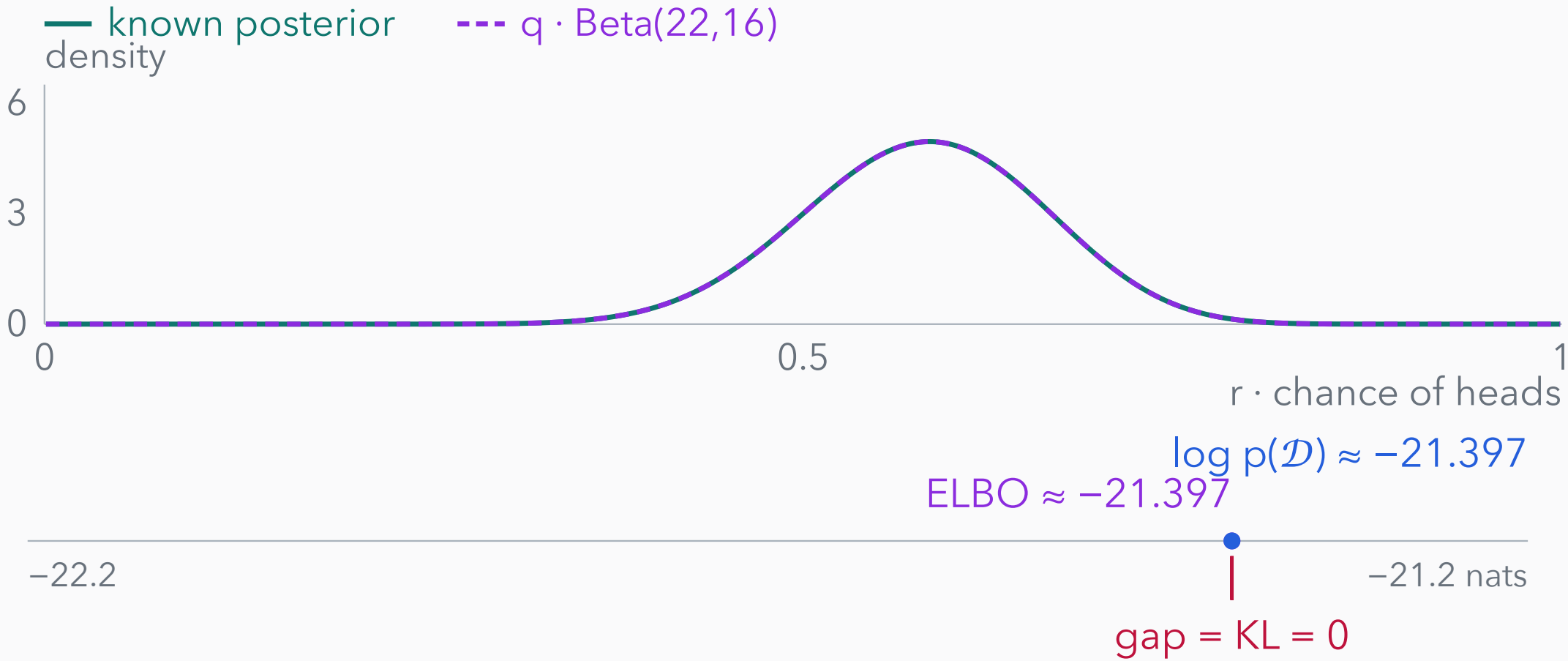
Add the ELBO and the gap. What is fixed?

Check the fixed total with three Beta candidates



Does the better candidate raise the ELBO by the amount its KL falls?

Check the fixed total with three Beta candidates



What happens at the exact posterior?

A mean must say which dimension it averages

object	shape	correct reduction
S scalar Beta draws	[S]	mean over S
S d-dimensional joint log densities	[S]	mean over S
S factorised coordinate log densities	[S, d]	sum over d, then mean over S

> Does every log_prob output still have an event dimension?

A mean must say which dimension it averages

object	shape	correct reduction
S scalar Beta draws	[S]	mean over S
S d-dimensional joint log densities	[S]	mean over S
S factorised coordinate log densities	[S, d]	sum over d, then mean over S

MultivariateNormal.log_prob already sums the event dimension. Do not sum the sample axis as well.

> What goes wrong if you sum over S before averaging?

An ordered sequence and a count have different evidence

$$p(H = 17 \mid r) = \binom{30}{17} r^{17}(1-r)^{13}$$

> Which event contains more possible sequences?

An ordered sequence and a count have different evidence

$$p(H = 17 \mid r) = \binom{30}{17} r^{17}(1-r)^{13}$$

observed object	log evidence
one ordered sequence	−21.397
only the head count	−2.796

> Does this constant change the posterior or its KL gap?

Check the sampled objective with analytic Beta moments

$$\mathbb{E}_q[\log r] = \mathrm{digamma}(a) - \mathrm{digamma}(a+b)$$

> Which two expectations determine the coin ELBO?

Check the sampled objective with analytic Beta moments

$$\mathbb{E}_q[\log r] = \mathrm{digamma}(a) - \mathrm{digamma}(a+b)$$

$$\mathbb{E}_q[\log(1-r)] = \mathrm{digamma}(b) - \mathrm{digamma}(a+b)$$

> Why write digamma rather than ψ here?

Check the sampled objective with analytic Beta moments

$$\mathbb{E}_q[\log r] = \mathrm{digamma}(a) - \mathrm{digamma}(a+b)$$

$$\mathbb{E}_q[\log(1-r)] = \mathrm{digamma}(b) - \mathrm{digamma}(a+b)$$

Insert these into expected log likelihood + expected log prior − expected log q.

> How was the identity checked non-circularly?